

Efficient Energy Tracking and Monitoring in IoT Enabled Smart Homes Using MQTT Protocol

A Project Report

submitted in partial fulfillment of the requirements for the award of the degree of

BACHELOR OF TECHNOLOGY

IN

COMPUTER SCIENCE & ENGINEERING

Submitted by

Ika. Sai Kumar

21761A0520

Chintalapudi. Joshna

21761A0514

Sigirisetty. Jyothika

21761A0554

Under the Esteemed guidance of

Mr. R. Ashok

Sr. Assistant Professor



DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

LAKIREDDY BALI REDDY COLLEGE OF ENGINEERING

(AUTONOMOUS)

Accredited by NAAC with 'A' Grade & NBA (ASE,CE,CSE,ECE,EEE,IT&ME)

An ISO 21001:2018, 14001:2015, 50001:2018 Certified Institution

Approved by AICTE, New Delhi and Affiliated to JNTUK, Kakinada

L.B. REDDY NAGAR, MYLAVARAM, NTR Dist., ANDHRA PRADESH – 521230

2021-2025



LAKIREDDY BALI REDDY COLLEGE OF ENGINEERING

(An Autonomous Institution since 2010)

Accredited by NAAC with 'A' Grade & NBA (Under Tier - I),

An ISO 21001:2018, 14001:2015, 50001:2018 Certified Institution

Approved by AICTE, New Delhi and Affiliated to JNTUK, Kakinada

L.B. REDDY NAGAR, MYLAVARAM, NTR DIST., A.P.-521 230.

hodcse@lbrce.ac.in, cseoffice@lbrce.ac.in, Phone: 08659-222 933, Fax: 08659-222931

DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING

CERTIFICATE

This is to certify that the project entitled “**Efficient Energy Tracking and Monitoring in IoT Enabled Smart Homes Using MQTT Protocol**” is being submitted by

Ika. Sai Kumar

21761A0520

Chintalapudi. Joshna

21761A0514

Sigirisetty. Jyothika

21761A0554

in partial fulfillment of the requirements for the award of degree of **B. Tech in Computer Science & Engineering** from **Jawaharlal Nehru Technological University, Kakinada** is a record of bonafide work carried out by them at **Lakireddy Bali Reddy College of Engineering**.

The results embodied in this Project report have not been submitted to any other University or Institute for the award of any degree or diploma

Mr. R. Ashok

(Sr. Assistant Professor)

Dr. D. Veeraiah

(Head of the Department)

EXTERNAL EXAMINER

ACKNOWLEDGEMENT

I extend my deepest appreciation to my project guide, **Mr. R. Ashok, Sr. Assistant Professor**, for their invaluable guidance, constant support, and insightful feedback throughout this project. Their encouragement and expertise have been instrumental in shaping my work.

I am also grateful to **Dr. K Devi Priya, Professor**, our project coordinator, for their continuous support and timely advice, which helped me stay on track and complete the project effectively.

I extend my heartfelt thanks to **Dr. D. Veeraiah**, Professor & Head of the Department, CSE for providing the necessary resources and a conducive environment for research and learning.

I am thankful to our respected **Principal, Dr. K. Appa Rao**, for creating an academic atmosphere that fosters innovation and knowledge-sharing.

I would also like to acknowledge the **teaching and non-teaching staff** of CSE department, whose guidance and assistance have been of great help throughout my academic journey.

A special thanks to my **family and friends** for their unwavering support, patience, and encouragement during the course of this project. Their belief in me has been a great source of motivation.

Lastly, I am grateful to everyone who has directly or indirectly contributed to the successful completion of this project.

I. Sai Kumar - 21761A0520

Ch. Joshna - 21761A0514

S. Jyothika - 21761A0554

DECLARATION

We hereby declare that the project report entitled **“Efficient Energy Tracking and Monitoring in IoT Enabled Smart Homes Using MQTT Protocol”** submitted to JNTUK in partial fulfillment of the requirements for the award of the **Bachelor of Technology** degree is an original work carried out by us. The content embodied in this project report is genuine and has not been submitted earlier to this university or any other university for the award of any degree, diploma, or prize.

Signatures of the students

I. Sai Kumar - 21761A0520

Ch. Joshna - 21761A0514

S. Jyothika - 21761A0554

ABSTRACT

The "Efficient Energy Tracking and Monitoring in IoT-Enabled Smart Homes Using MQTT Protocol" is a cutting-edge IoT solution designed to transform traditional households into intelligent, energy-efficient living spaces. Leveraging the powerful ESP32 microcontroller as the central hub, this system integrates multiple control mechanisms, including manual switches, MQTT-based remote control, and real-time energy monitoring. The project focuses on providing seamless control over home appliances while offering users detailed insights into energy consumption patterns.

By integrating cloud-based data logging through ThingSpeak and MQTT protocols, the system ensures reliable communication and analytics for energy usage and device management. The local LCD interface delivers instant feedback, enhancing accessibility and user experience. The combination of manual and automated control empowers users to operate appliances flexibly, catering to both tech-savvy and traditional preferences.

LIST OF CONTENTS	Pg.No
1. INTRODUCTION	1-6
1.1 Overview of the Project	1
1.2 Feasibility Study	2
1.3 Scope	4
2.LITERATURE SURVEY	7-9
2.1 Existing System and Its Drawbacks	7
3.SYSTEM ANALYSIS	10-14
3.1 Problem Definition	10
3.2 Proposed System and Its Advantages	10
3.3 System Requirements	12
3.4 Software and Tools Used	13
4.SYSTEM DESIGN	15-20
4.1 Overview of System Design	15
4.2 Architectural Design	16
5.IMPLEMENTATION	21-28
5.1 Coding Standards and Practices	21
5.2 Module Development	22
6.SYSTEM TESTING	29-30
6.1 Testing Strategies	29
7.RESULTS AND DISCUSSION	31-44
8.CONCLUSION AND FUTURE SCOPE	45-46
9.REFERENCES	47-50
10.APPENDICES	51-53

List of Figures

Figure No.	Figure Title	Pg.No
1.1	Smart Home with Appliances	2
4.1	System Architecture	17
4.2	Appliances connected to Smart Home Energy Management	19
7.1	Software Flowchart	33
7.2	Hardware Flowchart	34
7.3	Setup of the project	36
7.4	Mobile App With No Internet Connection	37
7.5	Mobile App With No Internet Connection with Appliances in off mode	37
7.6	Mobile App With Internet Connection with Appliances in on mode	38
7.7	Implementation of the Project with turn on appliance	38
7.8	LCD Display	39
7.9	ThingSpeak Websearch	40
7.10	Sign In Page	40
7.11	Channels Page	41
7.12	Channel Stats	41
7.13	Display of Charts	42
7.14	Fan Power Usage Chart	42
7.15	Bulb Power Usage Chart	43
7.16	Motor Power Usage Chart	43
7.17	Interface for Accessing Appliances	44

LIST OF ABBREVIATIONS

1. **IoT** - Internet of Things
2. **ESP32** - A microcontroller with built-in WiFi and Bluetooth
3. **MQTT** - Message Queuing Telemetry Transport
4. **LCD** - Liquid Crystal Display
5. **WiFi** - Wireless Fidelity
6. **GPIO** - General-Purpose Input/Output
7. **PIR** - Passive Infrared Sensor
8. **FreeRTOS** - Free Real-Time Operating System
9. **AI** - Artificial Intelligence
10. **EMS** - Energy Management System
11. **LED** - Light Emitting Diode
12. **AC** - Alternating Current
13. **DC** - Direct Current
14. **URL** - Uniform Resource Locator
15. **HTTP** - Hypertext Transfer Protocol
16. **HTTPS** - Hypertext Transfer Protocol Secure
17. **TLD** - Top-Level Domain
18. **CSV** - Comma-Separated Values
19. **JSON** - JavaScript Object Notation

1. INTRODUCTION

1.1 Overview of The Project

Enhancement of the smart grid is increasing every day to create a more robust and reliable system. The world is rapidly transitioning toward automation, leading to a rise in electricity consumption due to the excessive use of automated equipment. The reduction in fossil fuel availability and the increasing demand for electricity have resulted in a greater dependency on renewable energy sources (RES). While electricity production from RES is not the focus of this study, the efficient utilization of electricity through scheduling and coordination of appliances can significantly impacts overall consumption. According to previous research, the power sector is expected to witness a 38% increase in consumption by 2020, with the residential and commercial sectors experiencing a 16% upturn. Given this projected growth in electricity consumption, there is a pressing need to reorganize the pattern of energy consumption and production on both the demand and supply sides. The key approach to achieving this goal is the replacement of the traditional grid with a smart grid, integrated with the latest telecommunication technologies to enhance efficiency and reliability. One of the primary strategies for optimizing energy use in smart grids is load shifting, which involves moving electricity usage from peak hours to off-peak hours. This is achieved through techniques such as load clipping and valley filling. Load shifting helps minimize costs by scheduling appliances to operate during low-price periods. However, while it reduces electricity bills, it can lead to increased user discomfort due to constraints on appliance usage. To balance cost efficiency and user convenience, The system categorizes appliances into three groups: shiftable, non-shiftable, and controllable devices. By integrating scheduling strategies and remote control via a Node-RED-based UI, our approach optimizes energy consumption while maintaining user comfort. Additionally, power consumption is monitored using the ThinkSpeak platform, and the Aquila Optimization Algorithm (AO) is employed to achieve further efficiency gains. The proposed system aims to contribute to the sustainability of the electric grid by effectively managing household energy usage.

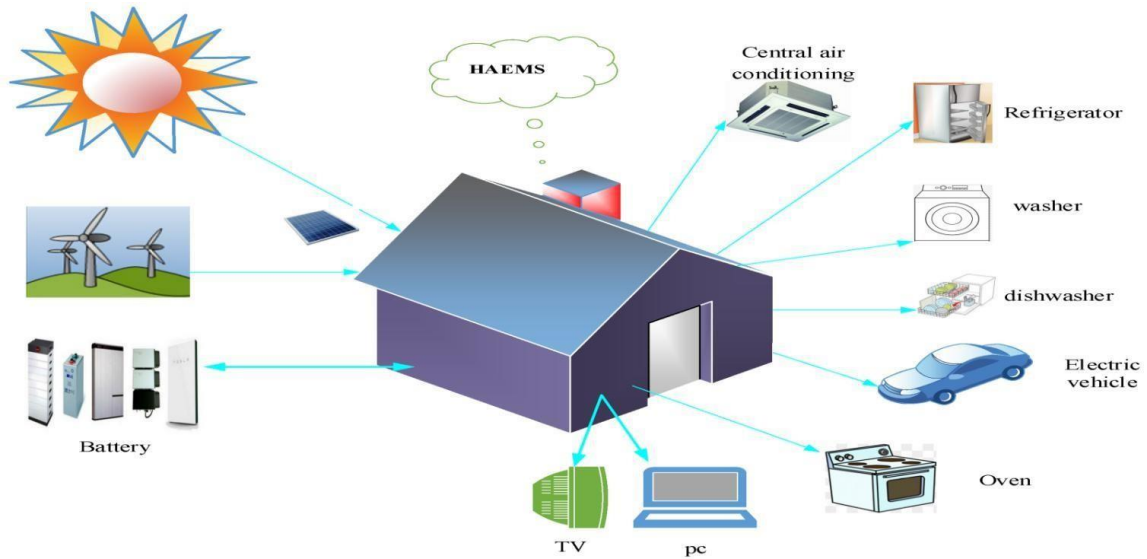


Figure 1.1 Smart Home with Appliances

1.2. Feasibility Study

The feasibility study confirms that the Advanced Multi-Device Home Automation and Energy Tracking System is technically viable, leveraging ESP32, MQTT, and ThingSpeak for seamless communication. It is economically feasible, utilizing cost-effective components while reducing energy consumption for long-term savings. The system is operationally efficient, offering an intuitive interface with manual and remote controls for diverse user accessibility. With a realistic deployment timeline, the project is scalable and adaptable for future smart home advancements.

1.2.1 Economical Feasibility

The system is highly cost-effective as it leverages low-cost microcontrollers like ESP32, free messaging through MQTT, and cloud-based data logging with ThingSpeak, which offers a free-tier option. Additionally, the use of smart plugs and relays eliminates the need to replace existing appliances, making the solution both affordable and practical. In terms of return on investment (ROI), households can benefit from long-term cost savings by optimizing energy consumption and reducing electricity bills. Furthermore, the system's scalability allows for commercial applications, creating potential revenue

opportunities in the smart home automation market.

1.2.2 Technical Feasibility

The system is technically feasible as it relies on well-established technologies like ESP32, MQTT, and ThingSpeak. The integration of manual switches, mobile applications (Blynk/Flutter), and Node-Red web control ensures ease of access.

Required Technologies:

- **ESP32 Microcontroller** – Manages communication and control.
- **MQTT Protocol** – Facilitates real-time messaging.
- **ThingSpeak** – Enables cloud-based data logging and visualization.
- **Node-Red** – Provides web-based control options.
- **Arduino IDE & FreeRTOS** – Used for development and multitasking.

Implementing the system presents several challenges, including ensuring seamless communication between multiple devices, which requires efficient synchronization of data and commands. Another critical aspect is optimizing FreeRTOS task scheduling to manage real-time monitoring effectively while maintaining system stability. Additionally, the project must address potential network latency and cloud service downtimes, as any disruptions in MQTT or ThingSpeak connectivity could impact real-time data updates and remote control functionality. Overcoming these challenges is essential for ensuring the system's reliability and efficiency.

1.2.3 Operational Feasibility

Designed for users of all technical backgrounds, featuring manual switches, mobile app, and LCD feedback. Real-time energy tracking enhances user engagement in energy conservation efforts.

The modular design allows easy updates and upgrades. The system can integrate additional sensors, automation features, and voice control in the future.

Potential Operational Risks:

- **Device Malfunctions** – ESP32 and relays must be properly tested for durability.
- **Data Security & Privacy** – MQTT communication should be secured to prevent

unauthorized access.

- **Power Supply Dependency** – Requires uninterrupted power or a backup power solution.

1.2.4 Schedule Feasibility

The estimated timeline for the deployment of the Advanced Multi-Device Home Automation and Energy Tracking System is approximately six months. The prototype development phase is expected to take 2-3 months, focusing on hardware integration, software development, and initial testing. This is followed by 1-2 months of testing and debugging, ensuring system stability, performance optimization, and troubleshooting any technical issues. Once the system is refined, a 1-month period is allocated for user trials and optimization, allowing for real-world testing, user feedback collection, and final adjustments. With this structured approach, the final deployment can be completed within six months, ensuring a robust and efficient smart home automation solution.

1.3 Scope

The "Advanced Multi-Device Home Automation and Energy Tracking with MQTT and with Data Logging" integrates IoT technology to enhance home automation with real-time energy monitoring. The system enables seamless control of traditional non-smart appliances through manual switches and remote MQTT-based control while also providing real-time tracking of voltage, current, power, and energy consumption. Using ThingSpeak, the system logs and visualizes energy data, helping users analyze their usage patterns for improved efficiency.

1.3.1 Key Areas of Scope

1.3.1.1 Smart Home Automation and Energy Monitoring

- The system enables automation and control of household appliances using esp32 microcontrollers, ensuring both remote and manual access.

- MQTT protocols allow users to operate multiple devices from anywhere, enhancing accessibility and convenience.
- Real-time energy tracking provides detailed insights into power consumption, helping users make data-driven decisions for efficient energy usage.

1.3.1.2 Multi-Device Control

- **Centralized Control & Reliable Communication:** The system uses the ESP32 **microcontroller** for centralized management and HiveMQ based MQTT for fast and reliable appliance communication.
- **Multi-Appliance Management:** Supports simultaneous control of multiple devices, ensuring efficient and smooth operation of home appliances.
- **Customizable Automation:** Users can create customizable device groups to control multiple appliances with a single command and set up scheduled automation for routine tasks like turning lights or fans on/off at specific times.
- **Future AI Integration:** The system has the potential for voice command integration with AI assistants like Alexa and Google Assistant for a more seamless smart home experience.

1.3.1.3 User-Friendly Interfaces

- Supports Blynk/Flutter-based mobile applications for easy and remote access to appliance control.
- Provides an LCD interface displaying real-time energy consumption and appliance status for instant user insights.
- Uses Node-RED to enable cross-platform accessibility, allowing users to monitor and control appliances via a web interface.
- Ensures easy operation for users of all technical backgrounds, making home automation accessible to everyone.

1.3.1.4 Seamless Communication & Data Processing

- Utilizes FreeRTOS for efficient task scheduling and multitasking, ensuring

smooth system performance.

- Implements MQTT protocol for fast and secure communication between devices.
- Integrates Node-RED for web-based monitoring and appliance control from any device.
- Designed for scalability and low latency, enabling seamless communication across multiple devices.

1.3.1.5 Cost-Effectiveness & Future Scalability

- Uses smart plug-based solutions to control non-smart appliances without requiring expensive replacements.
- Reduces the need for costly smart device upgrades by integrating IoT functionalities into traditional appliances.
- Designed to support additional smart home features, including AI-based automation and predictive control.
- Enables data-driven insights to optimize energy consumption, promoting efficient and sustainable home automation.

2. LITERATURE SURVEY

2.1 Existing System and its Drawbacks

2.1.1 Existing Systems for Phishing Detection

In traditional home automation and energy management systems, several methodologies have been used to control appliances and monitor energy consumption. However, these existing systems have several limitations:

2.1.1.1 Limited Control Mechanisms

- Most conventional systems rely on either manual switches or basic remote control, lacking a unified approach for managing both smart and non-smart appliances.
- The absence of cross-platform support makes it difficult to operate devices remotely via mobile apps or web interfaces.
- Existing systems lack scheduling features, preventing users from automating routine tasks such as turning appliances on/off at specific times.
- Most traditional setups do not support voice commands or AI-based assistants like Alexa or Google Assistant, reducing ease of use.

2.1.1.2 Lack of Real-Time Energy Monitoring

- Existing home automation solutions often do not provide real-time tracking of voltage, power, or energy consumption.
- Without detailed energy usage analytics, users cannot make data-driven decisions to optimize power consumption.
- No predictive energy management, making it difficult to detect unusual consumption patterns or prevent energy wastage.
- Lack of historical data storage and visualization, preventing users from analyzing long-term trends in power consumption.

2.1.1.3 Inefficient Communication and Integration

- Many older systems use legacy communication protocols, which are not optimized for IoT-based automation.
- The lack of MQTT-based messaging results in delayed or unreliable communication between devices.
- Limited interoperability with modern IoT platforms, making it difficult to integrate multiple smart home devices seamlessly.
- High latency and connectivity issues, leading to slow response times when controlling appliances remotely.

2.1.1.4 High Cost & Scalability Issues

- Many smart home solutions require expensive device upgrades, making them less affordable for households with traditional appliances.
- Most systems do not offer modular scalability, restricting future enhancements like AI-based automation and voice control integration.
- High initial investment and maintenance costs discourage widespread adoption of smart home technologies.
- Incompatibility with existing appliances forces users to replace functional devices rather than integrating them into an IoT-based system.

2.1.2 Drawbacks

- Limited to manual or basic remote control; lacks unified control for both smart and non-smart appliances.
- No support for scheduling or automation of routine appliance tasks.
- Absence of real-time energy monitoring (voltage, current, power, and consumption).
- No analytics or historical data visualization for optimizing energy usage.
- Uses outdated communication protocols; lacks MQTT support.
- High latency and unreliable remote device control.

- Incompatibility with modern IoT platforms and devices.
- Requires costly smart device upgrades; not budget-friendly for traditional appliance users.
- Poor scalability; difficult to integrate new features like AI or voice assistants.
- Lack of data security features and cloud connectivity in traditional systems.

SYSTEM ANALYSIS

3.1 Problem Defintion

With the rapid growth in home automation, energy consumption is also rising significantly. Traditional energy systems and manual appliance control methods lack the intelligence and flexibility needed to optimize electricity usage in modern homes. Additionally, most homes still use non-smart appliances that cannot be controlled remotely or integrated into energy-saving schemes. Smart homes powered by IoT can address these challenges by enabling real-time control, monitoring, and automation of appliances. However, existing systems face limitations such as high costs, limited support for legacy devices, and inefficient communication protocols. These shortcomings reduce their usability, scalability, and effectiveness.

3.2 Proposed System and its Advantages

The proposed "Advanced Multi-Device Home Automation and Energy Tracking with MQTT and with Data Logging" integrates ESP32 microcontrollers to provide a unified control mechanism for both smart and non-smart appliances, allowing seamless automation through manual switches, mobile apps, and MQTT-based remote control. The system implements real-time energy monitoring, tracking voltage, current, power, and energy consumption, with data logging and visualization via ThingSpeak for improved analytics and optimization. To ensure efficient multi-device communication, it leverages the MQTT protocol (HiveMQ) for fast, reliable, and low-latency data exchange between connected appliances, enabling smooth and responsive operations.

For user accessibility, the system supports mobile app integration (Blynk/Flutter), Node-RED-based web control, and an LCD interface, providing real-time feedback on appliance status and energy usage. It is designed as a cost-effective and scalable solution, allowing users to control non-smart appliances using smart plugs and relays, thus eliminating the need for costly device replacements. The system also employs FreeRTOS for optimized multitasking, ensuring efficient task handling across MQTT communication, GPIO control, and ThingSpeak updates for real-time data processing.

Further more, the system is built with future scalability and sustainability in mind, supporting AI-based automation, voice control integration (Alexa/Google Assistant), and predictive energy management to promote a smarter, more energy-efficient home environment.

3.2.1 Key Features of the Proposed System:

- **Centralized Control:** Uses ESP32 microcontrollers to manage both smart and non-smart appliances through manual switches, mobile apps, and MQTT-based remote control.
- **Real-Time Energy Monitoring:** Tracks voltage, current, power, and energy consumption, with data logging and visualization using ThingSpeak for detailed analytics.
- **Reliable Multi-Device Communication:** Implements MQTT protocol (HiveMQ) for fast, secure, and low-latency communication between connected appliances.
- **User-Friendly Interface:** Supports mobile app integration (Blynk/Flutter), Node-RED-based web control, and an LCD display for real-time feedback on appliance status and energy usage.
- **Cost-Effective and Scalable:** Enables IoT compatibility for non-smart appliances using smart plugs and relays, reducing the need for expensive smart device upgrades.
- **Optimized Task Management:** Utilizes FreeRTOS for efficient multitasking, ensuring smooth execution of MQTT communication, GPIO control, and ThingSpeak updates.
- **AI-Based Automation & Voice Control:** Designed to support future enhancements like AI-based automation, predictive energy management, and voice control integration with Alexa/Google Assistant.

3.2.2 Advantages of the Proposed System:

- **Seamless Smart and Non-Smart Appliance Control:** Provides a unified control mechanism for both smart and traditional appliances using ESP32, MQTT, and smart relays.
- **Real-Time Energy Insights:** Tracks voltage, current, power, and energy consumption, allowing users to monitor and optimize energy usage efficiently.
- **Remote Accessibility & Control:** Enables mobile app (Blynk/Flutter) and web-based (Node-RED) access, allowing users to control appliances from anywhere.
- **Fast & Reliable Communication:** Uses MQTT protocol (HiveMQ) for low-latency, secure, and efficient data exchange, ensuring seamless device interactions.
- **Cost-Effective & Easy Integration:** Eliminates the need for expensive smart devices by making existing appliances IoT-compatible with smart plugs and relays.
- **Scalability & Future Expansion:** Designed for modular expansion, supporting future enhancements like AI-driven automation, predictive energy management, and voice control integration.
- **User-Friendly Interface:** Features an LCD display for real-time feedback, ensuring accessibility for users of all technical backgrounds.

3.3 SYSTEM REQUIREMENTS

3.1.1 Software Requirements

- **Arduino IDE** – Used for programming and flashing the ESP32 microcontroller.
- **FreeRTOS** – Handles multitasking for efficient processing of system operations.
- **MQTT (HiveMQ)** – Enables fast and secure communication between devices.
- **ThingSpeak** – Cloud-based platform for data logging, analysis, and visualization.
- **Node-RED** – Provides a web-based dashboard for monitoring and controlling appliances remotely.
- **Blynk/Flutter** – Used to develop the mobile application for remote access and control.

3.1.2 Hardware Requirements

- **ESP32 Microcontroller** – Acts as the central hub for managing appliance control

and communication.

- **Relay Modules** – Enable switching of non-smart appliances based on commands.
- **Voltage and Current Sensors** – Monitor power consumption in real time.
- **LCD Display** – Provides instant feedback on energy usage and system status.
- **Manual Switches** – Allow traditional control alongside automated IoT-based operations.
- **Wi-Fi Router** – Ensures stable internet connectivity for cloud-based monitoring and remote control.
- **Power Supply Modules** – Provide necessary voltage and current for the system component.

3.4 Software Used in The Project

The “Advanced Multi-Device Home Automation and Energy Tracking System” utilizes various software tools and platforms to enable efficient system development, communication, and real-time monitoring. These software components ensure seamless integration, automation, and remote accessibility for the system.

- **Arduino IDE:** Used for programming the ESP32 microcontroller, writing firmware, and uploading code to control appliances and manage sensor data.
- **FreeRTOS:** A real-time operating system used for efficient task scheduling and multitasking, ensuring smooth execution of multiple system processes.
- **MQTT (HiveMQ):** A lightweight messaging protocol that facilitates fast and reliable communication between the ESP32 and connected devices.
- **ThingSpeak:** A cloud-based data visualization platform used to log and analyze real-time energy monitoring data.
- **Node-RED:** A web-based IoT platform that provides an interactive dashboard for remote monitoring and control of appliances.
- **Blynk/Flutter:** A mobile application framework used to develop the smartphone app for remote appliance control and system monitoring

These software tools work together to create a robust, scalable, and user-friendly home automation solution, enabling efficient energy tracking, remote device management, and

future scalability.

Development Environment

- **Hardware-Software Integration:** The system is developed using the Arduino IDE for programming the ESP32 microcontroller, with FreeRTOS managing task execution and MQTT (HiveMQ) ensuring reliable device communication.
- **Cloud and User Interface Development:** ThingSpeak is used for real-time data logging and visualization, while Node-RED enables web-based monitoring and control, and Blynk/Flutter supports mobile app development for remote accessibility.

Data Processing and Visualization

- The “Advanced Multi-Device Home Automation and Energy Tracking with MQTT and with Data Logging” incorporates efficient data processing and visualization techniques to provide real-time insights into appliance control and energy consumption. The system collects sensor data from ESP32, including voltage, current, power, and energy consumption metrics, which are then processed and transmitted via MQTT (HiveMQ) to ensure fast and secure communication.
- For data storage and analysis, the system utilizes ThingSpeak, a cloud-based IoT analytics platform, where energy data is logged, processed, and visualized through graphs, charts, and real-time dashboards. Additionally, Node-RED provides a web-based interface, enabling users to remotely monitor appliance status and energy trends. The mobile application (Blynk/Flutter) further enhances visualization by offering a user-friendly dashboard for tracking energy consumption and controlling devices from anywhere. This approach ensures that users receive accurate, real-time insights into their power usage patterns, allowing for efficient decision-making and energy optimization.

4. SYSTEM DESIGN

4.1 Overview of System Design

The "Advanced Multi-Device Home Automation and Energy Tracking System" is designed to provide seamless control, real-time energy monitoring, and efficient communication for smart and non-smart appliances. The system follows a modular architecture, integrating hardware, software, and cloud-based components to ensure reliability and scalability.

At the core of the system is the ESP32 microcontroller, which serves as the central hub, handling appliance control, sensor data processing, and communication with remote interfaces. The system uses MQTT (HiveMQ) for fast and secure messaging, enabling users to control devices remotely via mobile apps (Blynk/Flutter) and a web-based dashboard (Node-RED). Additionally, real-time energy tracking is implemented using voltage and current sensors, with data logged and visualized through ThingSpeak for analysis. To ensure efficient multitasking, the system is built on FreeRTOS, allowing simultaneous execution of appliance control, MQTT communication, and energy data processing. The design also includes manual switches and an LCD display, providing users with traditional control options and real-time status feedback. The architecture is cost-effective, scalable, and future-ready, with potential for AI-driven automation and voice control (Alexa/Google Assistant) integration.

The system design of the "Advanced Multi-Device Home Automation and Energy Tracking with MQTT and with Data Logging" is structured to provide seamless control, real-time monitoring, and efficient communication for home automation. The architecture is built around the ESP32 microcontroller, which acts as the central hub, integrating manual switches, mobile apps (Blynk/Flutter), and web-based (Node-RED) interfaces for remote accessibility. To ensure efficient data transmission and device communication, the system leverages the MQTT protocol(HiveMQ) for fast, secure, and reliable operations. Additionally, real-time energy tracking and analytics are implemented using ThingSpeak, allowing users to monitor voltage, current, power, and energy consumption, helping them

make data-driven decisions for optimized energy usage.

- **Centralized Control & Multi-Platform Access:** The system integrates ESP32, manual switches, mobile apps, and web interfaces (Node-RED) to provide seamless remote and manual control over appliances.
- **Efficient Communication & Energy Monitoring:** Uses MQTT (HiveMQ) for fast and secure device communication and ThingSpeak for real-time energy tracking and data visualization.
- **Scalability & Future Enhancements:** Designed to support AI-driven automation, predictive energy management, and voice control (Alexa/Google Assistant), ensuring adaptability for future smart home expansions.

4.2 Architecture Design:

The circuit diagram shown in the image represents a Home Automation System based on the ESP32 microcontroller. It is designed to control multiple electrical appliances, such as light bulbs and a fan, using a relay module. The system integrates manual switches for local control and an IoT-based remote control system via MQTT or Wi-Fi.

Components in the Circuit:

4.2.1. ESP32 Microcontroller

- The ESP32 is the main controller in the circuit.
- It connects to a Wi-Fi network and receives commands for controlling home appliances.
- It sends signals to the relay module to turn devices ON or OFF.
- The ESP32 is powered by a 5V external power supply.

4.2.2. 4-Channel Relay Module

- The relay module acts as an electromechanical switch that controls high-voltage devices.
- It receives signals from the ESP32 GPIO pins to turn ON/OFF the connected bulbs and fan.

- Each relay operates as a switch, allowing the circuit to control multiple devices.

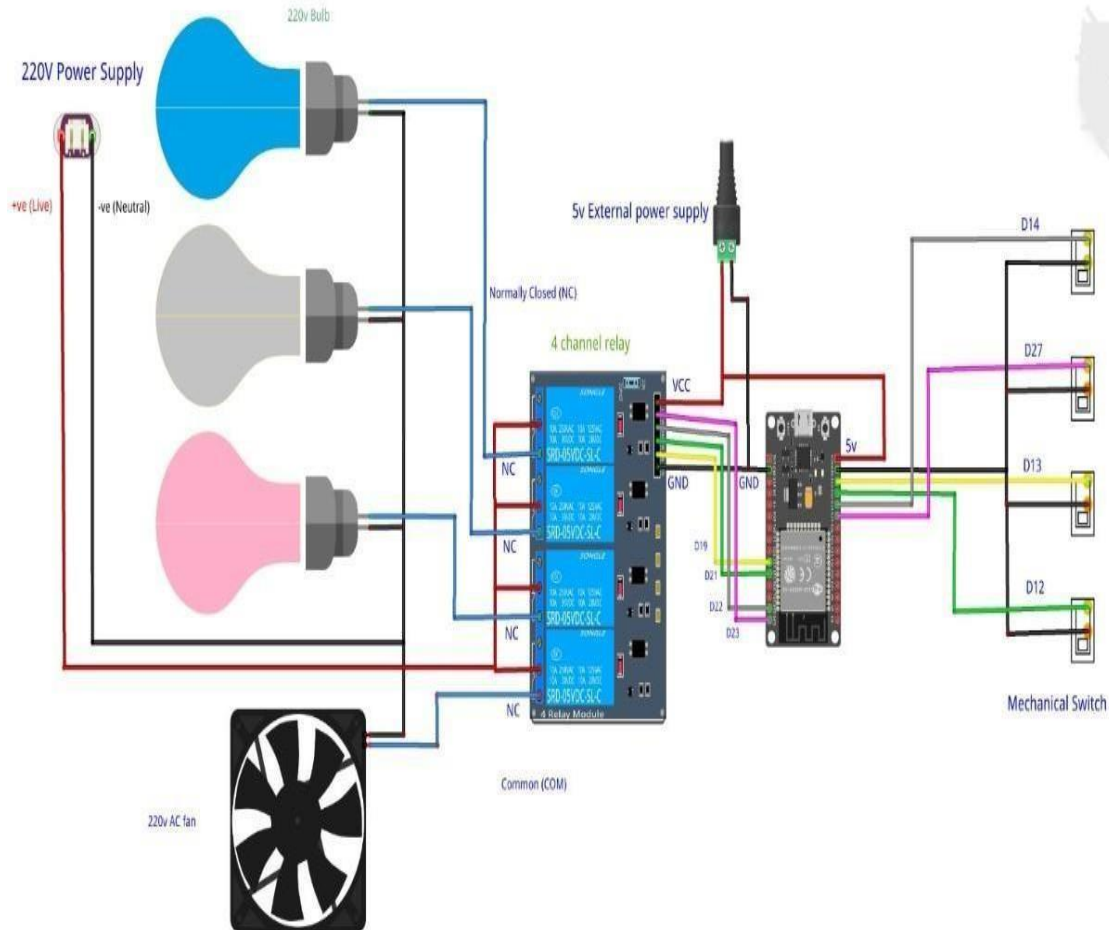


Figure 4.1 System Architecture

4.2.3. 220V AC Power Supply

- This AC power supply provides electricity to the connected light bulbs and fan.
- The relay module acts as an intermediary between the ESP32 and high-voltage devices.
- The Neutral (-ve) wire is directly connected to the appliances, while the Live (+ve) wire is connected through the relay.

4.2.4. Light Bulbs (3)

- These represent the loads in the circuit, controlled via the relay module.
- Each bulb is connected to a separate relay channel, allowing individual control.

4.2.5. 220V AC Fan

- The fan is another high-voltage device controlled via the relay module.
- It is connected similarly to the bulbs, using a separate relay channel.

4.2.6. Mechanical Switches

- These manual switches provide an alternative control option.
- They are connected to the ESP32 GPIO pins (D14, D27, D13, D12).
- When a user presses a switch, the ESP32 detects the input and toggles the corresponding relay.

4.2.7. 5V External Power Supply

- Powers the ESP32 microcontroller and relay module.
- Ensures stable operation without relying on USB power.

4.2.8 Working of the Circuit:

1. The ESP32 initializes and connects to the Wi-Fi network.
2. The user can send ON/OFF commands via MQTT or a mobile application.
3. The ESP32 processes the commands and activates the corresponding relay.
4. The relay switches ON/OFF the bulb or fan by controlling the 220V AC power supply.
5. If a manual switch is pressed, the ESP32 detects it and toggles the respective appliance.
6. Real-time updates are sent to the cloud for energy tracking (if integrated).

Advantages

- Remote control via Wi-Fi (IoT-based)
- Manual override using mechanical switches
- Energy-efficient home automation
- Multiple appliance control using a single microcontroller
- Scalable design for adding more devices in the future

4.2.9 Appliances:

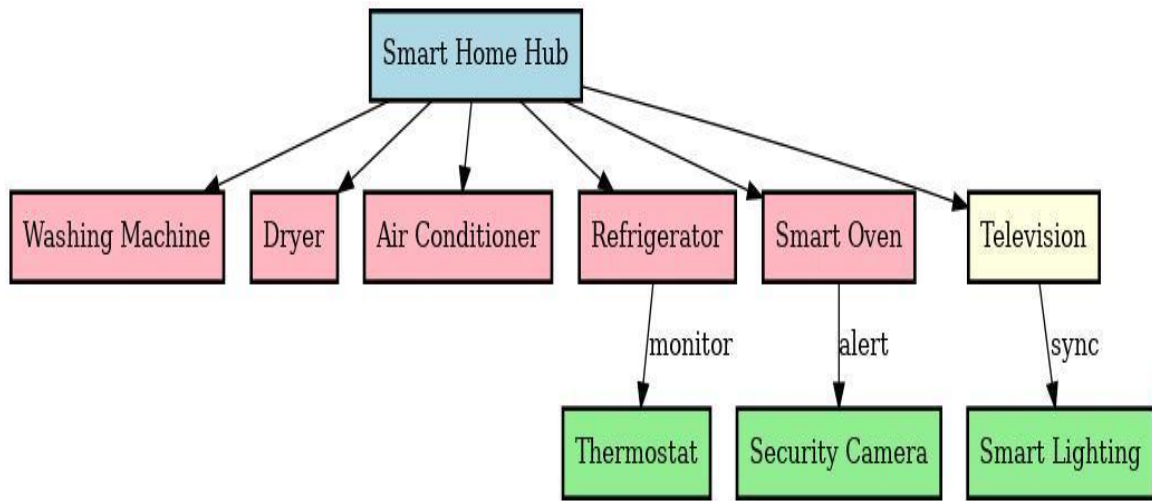


Figure 4.2 Appliances connected to Smart Home Energy Management

The system architecture consists of several key components working together to enable efficient control and monitoring of appliances. At the core of the system is the ESP32 microcontroller, which serves as the central processing unit, managing the operation of all connected devices. A relay module acts as an interface between the ESP32 and the appliances, allowing digital control of AC-powered devices. Wireless communication is facilitated by a WiFi module, enabling seamless interaction between the ESP32 and the laptop hosting the user interface. The control flow and user interface are developed using Node-RED, a graphical programming environment. Additionally, the ThingSpeak platform is integrated to collect and visualize power consumption data in real-time.

4.2.10 Optimization Algorithm:

To further enhance the energy efficiency of our system, we implemented the Aquila Optimization (AO) algorithm. AO is a metaheuristic algorithm inspired by the hunting behavior of eagles (Aquila species) in nature. It is characterized by a balance between exploration and exploitation phases to efficiently search for the optimal solution. The Aquila Optimization algorithm models four hunting strategies eagles. The mathematical

model of the AO algorithm can be represented by the following equations.

1. High Soaring as Exploration Phase

$$1(+1) = \text{best}() \times (1 -) + \text{rand}() \times$$

1. Prey Detection as Transition Phase

$$2(+1) = \text{best}() - \times \text{mean}()$$

2. Attacking as Exploitation Phase

$$3(+1) = \text{best}() - \times |\text{best}() \times 2 \times \text{rand}() - ()$$

3. Cooperative hunting as Local Search Space

$$4(+1) = F \times \text{best}() - (1 \times () \times \text{rand}()) - (2 \times \text{Levy}())$$

Where $X(t)$ is the current position of the search agent at iteration t . $X \text{ best}(t)$ is the position of the best solution found so far. $X \text{ rand}(t)$ is a randomly selected solution. $X \text{ mean}(t)$ is the mean position of all solutions. T is the maximum number of iterations M , y , QF , G_1 , and G_2 are control parameters $\text{Levy}(D)$ is the Lévy flight function. In this system, AO is applied to determine the best schedule for operating appliances while minimizing energy costs and maintaining user comfort. The optimization process considers multiple constraints, including peak-hour restrictions, total energy consumption limits, and appliance-specific operating requirements. Additionally, the system includes firmware for the ESP32, written in Micro Python or communication with Node-RED and ThinkSpeak, and relay control. The firmware ensures real-time response and secure communication through encryption techniques.

5.CODING & IMPLEMENTATION

5.1 Coding Standards and Practices

To ensure maintainability, reliability, and scalability of the system, the development of the "Efficient Energy Tracking and Monitoring in IoT Enabled Smart Homes" project followed standard programming conventions and best practices. The coding strategy focuses on modularity, readability, error handling, and security, ensuring a professional and production-ready codebase.

Key Practices Followed:

- **Modular Code Structure:**
 - The project is divided into multiple functional modules (e.g., Wi-Fi config, MQTT, energy monitoring, LCD updates).
 - Each module is independent, making debugging and maintenance easier.
- **Consistent Naming Conventions:**
 - Variables, functions, and topics use camelCase or underscore_case as per convention.
 - Example: energyMonitoringTask(), mqttReconnect(), gpioState1.
- **Code Reusability:**
 - Functions like updateLCD(), callback(), and reconnect() are reused across tasks.
 - Promotes DRY (Don't Repeat Yourself) principle.
- **Error Handling:**
 - Reconnection mechanisms are implemented for MQTT and Wi-Fi disconnections.
 - Failsafe behavior ensures the system continues functioning in adverse conditions.
- **Commenting and Documentation:**
 - Each function and critical code block is commented.
 - Explains logic, expected inputs/outputs, and error scenarios.
- **Task Prioritization (FreeRTOS):**

- Uses proper task priorities for concurrent processes like energy monitoring, data publishing, and GPIO control.
- Ensures the system remains responsive under load.
- **Security Practices:**
 - Sensitive information such as MQTT credentials and topics are stored securely.
 - Encapsulation of network and cloud credentials into dedicated files or variables.

5.2. Module Development

The implementation of the system was carried out using a **modular development approach**, where each functional unit was independently developed, tested, and then integrated into the main system. This ensured clarity in development, efficient debugging, and flexibility in upgrading specific features.

Developed Modules:

1. Wi-Fi Configuration Module

- Uses WiFiManager to connect to existing networks.
- Supports fallback AP mode for easy reconfiguration.
- Auto-reconnect functionality ensures system uptime.

2. MQTT Communication Module

- Handles real-time messaging with HiveMQ and ThingSpeak.
- Subscribes to control topics and publishes energy data.
- Uses PubSubClient for lightweight and efficient communication.

3. GPIO Control Module

- Interfaces with relays to control AC appliances.
- Reads manual switch inputs.
- Includes debounce logic to prevent switch misreads.

4. Energy Monitoring Module

- Uses EmonLib to measure voltage and current.
- Computes power and energy in real-time.
- Validates values and forwards to LCD and ThingSpeak.

5. LCD Display Module

- Displays real-time voltage, current, and energy consumption.
- Provides visual feedback to users during operation.
- Updates every second without delay to other tasks.

6. ThingSpeak Data Logging Module

- Periodically publishes sensor and control data to the cloud.
- Visualizes energy usage trends via ThingSpeak dashboards.
- Synchronizes with Node-RED and mobile apps.

7. Mobile App and Node-RED Web Interface Module

- Allows remote appliance control and monitoring.
- Built using Blynk/Flutter and Node-RED UI.
- Reflects switch states in real-time and enables toggling.

5.2.1 Library Inclusions and WiFi Configuration

This section includes necessary libraries for handling LCD, MQTT, and energy monitoring.

```
#include "EmonLib.h"
```

```
#include <LiquidCrystal_I2C.h>
#include <WiFiManager.h>
#include <PubSubClient.h>
```

The **WiFiManager** library is used to create an access point for easy WiFi configuration.

5.2.2 MQTT Configuration

Defines MQTT broker settings for **HiveMQ** (for device control) and **ThingSpeak** (for energy monitoring).

```
const char* mqttServer = "broker.hivemq.com";
const int mqttPort = 1883;
const char* mqttUser = "123456789@1";
const char* mqttPassword = "123456789@1";

const char* tsMqttServer = "mqtt3.thingspeak.com";
const int tsMqttPort = 1883;
const char* tsMqttUser = "Ni4yPBUCJxw1CRApJjMdOh0";
const char* tsMqttPassword = "4omwrgqhu/5wUJH7Fm3nYNRe";
const char* tsClientID = "Ni4yPBUCJxw1CRApJjMdOh0";
```

5.2.3 GPIO Pin Configuration

Defines output relays and input buttons for appliance control.

```
const int GPIO0 = 13;
const int GPIO1 = 4;
const int GPIO2 = 5;
const int GPIO3 = 12;
const int Button0 = 14;
const int Button1 = 27;
const int Button2 = 26;
```



```
const int Button3 = 25;
```

5.2.4 MQTT Callback Function

This function listens to MQTT messages and updates the appliance states accordingly.

```
void callback(char* topic, byte* payload, unsigned int length) {  
    String message = "";  
    for (unsigned int i = 0; i < length; i++) {  
        message += (char)payload[i];  
    }  
    message.trim();  
    if (String(topic) == topic_gpio0) {  
        gpioState0 = (message == "1");  
        digitalWrite(GPIO0, gpioState0);  
    } else if (String(topic) == topic_gpio1) {  
        gpioState1 = (message == "1");  
        digitalWrite(GPIO1, gpioState1);  
    } else if (String(topic) == topic_gpio2) {  
        gpioState2 = (message == "1");  
        digitalWrite(GPIO2, gpioState2);  
    } else if (String(topic) == topic_gpio3) {  
        gpioState3 = (message == "1");  
        digitalWrite(GPIO3, gpioState3);  
    }  
}
```

5.2.5 WiFi and MQTT Connection

Ensures a stable connection to the MQTT broker and reconnects if disconnected.

```
void reconnect(PubSubClient& mqttClient, const char* user, const char* password, const
char* clientID = nullptr) {
    while (!mqttClient.connected()) {
        if (mqttClient.connect(clientID ? clientID : "espClient", user, password)) {
            if (&mqttClient == &client) {
                mqttClient.subscribe(topic_gpio0);
                mqttClient.subscribe(topic_gpio1);
                mqttClient.subscribe(topic_gpio2);
                mqttClient.subscribe(topic_gpio3);
            }
        } else {
            delay(5000);
        }
    }
}
```

5.2.6 LCD Update Function

Displays voltage, current, and energy usage on the LCD screen.

```
void updateLCD(float Vrms, float Irms) {
    lcd.clear();
    lcd.setCursor(0, 0);
    lcd.printf("V:%.2f I:%.2f", Vrms, Irms);
    lcd.setCursor(0, 1);
    lcd.printf("E:%.3fkWh", energyConsumed / 1000.0);
}
```

5.2.7 Energy Monitoring Task

Uses the EmonLib library to measure voltage and current, then calculates energy consumption.

```
void energyMonitoringTask(void* pvParameters) {  
    while (true) {  
        emon.calcVI(20, 2000);  
        float Vrms = emon.Vrms;  
        float Irms = emon.Irms;  
  
        if (Vrms >= 150.0 && Irms >= 0.01) {  
            powerValue = Vrms * Irms;  
            energyConsumed += (powerValue / 3600.0);  
        }  
        updateLCD(Vrms, Irms);  
        delay(1000);  
    }  
}
```

5.2.8 ThingSpeak Data Logging

Sends power usage data to ThingSpeak every 20 seconds.

```
void thingSpeakTask(void* pvParameters) {  
    while (true) {  
        if (!tsMqttClient.connected()) {  
            reconnect(tsMqttClient, tsMqttUser, tsMqttPassword, tsClientID);  
        }  
        tsMqttClient.loop();  
  
        char payload[100];
```

```
    snprintf(payload, sizeof(payload),  
    "field1=%.2f&field2=%d&field3=%d&field4=%d&field5=%d",  
        powerValue, gpioState0, gpioState1, gpioState2, gpioState3);  
  
    tsMqttClient.publish(tsPublishTopic, payload);  
    delay(20000);  
}
```

The implementation combines IoT, cloud computing, and real-time energy monitoring to create an efficient and user-friendly home automation system. By integrating MQTT communication, local button controls, and real-time data visualization, the system offers reliability, flexibility, and remote accessibility, making it a practical solution for modern smart homes.

6.SYSTEM TESTING

6.1 Testing Strategies

Testing is a crucial phase in the software and hardware development lifecycle, ensuring that a system meets the specified requirements and functions correctly. It involves evaluating a system's components to detect defects, improve performance, and verify compliance with industry standards. Testing can be performed manually or through automation, depending on the complexity of the system and the project requirements. Testing is essential to ensure the reliability, security, efficiency, and usability of a system. It helps identify bugs, vulnerabilities, and performance bottlenecks before deployment, reducing the chances of failures in real-world scenarios. The scope of testing is broad, covering functional, non-functional, and maintenance aspects. Testing is the process of evaluating a system or component to verify that it meets the required specifications and functions correctly. It helps identify defects, improve performance, and ensure reliability before deployment. Testing can be conducted manually or using automated tools, depending on the requirements.

6.1.1 Types of Tests

6.1.1.1 Remote Testing

- Performed from a remote location without physical access to the system.
- Commonly used for cloud applications, IoT devices, and web-based platforms.
- Ensures proper functionality across different network environments.
- Can be automated to monitor real-time system behaviour.
- Helps in identifying latency issues and compatibility concerns.
- Reduces testing costs by eliminating the need for on-site presence.
- Examples: Cloud-based testing platforms like Browser Stack, Sauce Labs.

6.1.1.2 Manual Testing

- Conducted by human testers without automation tools.
- Used to find usability issues, design flaws, and functional errors.
- Requires tester experience and creativity to explore various scenarios.
- Essential for small-scale projects and exploratory testing.
- Helps verify system performance under real-world user interactions.
- Detects visual and layout inconsistencies across devices.
- Involves different phases like test case creation, execution, and defect reporting.

6.1.1.3 Power Visualization Testing

- Evaluates the power consumption and energy efficiency of a system.
- Used in embedded systems, IoT devices, and smart appliances.
- Helps optimize power usage by analyzing consumption patterns.
- Prevents overheating and power surges in electronic components.
- Visualizes data through graphs, charts, and simulation tools.
- Ensures compliance with energy efficiency standards.
- Reduces operational costs by improving power management.

6.1.1.4 Importance of Testing

- Ensures the system functions as expected in different environments.
- Detects defects and prevents failures before deployment.
- Improves software performance, reliability, and security.
- Helps optimize power usage and reduce operational costs.
- Ensures compliance with industry standards and regulations.

By implementing remote, manual, and power visualization testing, organizations can enhance the quality, efficiency, and reliability of their systems.

7.RESULTS AND DISCUSSION

The implementation of the “Efficient Energy Tracking and Monitoring in IoT Enabled Smart Homes Using MQTT Protocol” successfully met all design objectives. The system demonstrated real-time energy monitoring, remote control of appliances, cloud-based logging, and dual control options (manual and mobile/web interface). The integration of ESP32, MQTT, ThingSpeak, Node-RED, and the Aquila Optimization Algorithm delivered a robust and scalable energy management system for smart homes.

Real-Time Energy Monitoring:

- Voltage, current, and power consumption data were captured using sensors and displayed on an LCD.
- Energy usage data was logged every 20 seconds to ThingSpeak for visualization and analysis.

Remote Appliance Control:

- Appliances were toggled via mobile app (Blynk/Flutter), Node-RED web interface, and physical switches.
- MQTT ensured fast, reliable control from remote locations.

Cloud Integration:

- ThingSpeak visualized trends in power consumption over time, including separate graphs for individual appliances (fan, bulb, motor).

Task Scheduling and Optimization:

- The system used FreeRTOS for efficient multitasking and scheduling.
- The Aquila Optimization Algorithm helped suggest optimal time slots to operate appliances, minimizing power costs and peak load issues.

System Responsiveness:

- Response time to MQTT commands was consistently under 1 second.
- The system maintained reliable performance during extended operation and network fluctuations.

ThingSpeak Dashboards revealed:

- Time-based fluctuations in energy consumption.
- Peaks during high appliance usage hours.
- Reduced energy usage during scheduled automation periods.

Node-RED Dashboard:

- Provided a real-time control panel with switch statuses and appliance logs.
- Integrated energy usage graphs for better decision-making.

The Hybrid Home Control System was successfully implemented and tested, demonstrating its capability to control home appliances via multiple interfaces, including voice commands (Google Assistant), manual switches, and an MQTT-based mobile app. The system efficiently integrated ESP32, MQTT, and real-time energy monitoring to provide seamless automation.

7.1 Software System Workflow:

1. **Initialization** - The system initializes by setting up required parameters, including Wi-Fi, MQTT, and sensor configurations.
2. **Data Collection from Appliances Network** - The ESP32 collects real-time data from connected appliances.
3. **Data Verification** - If data is received, it is processed further; otherwise, the system waits for new data.
4. **Display Data on LCD** - The collected data is displayed on the LCD screen for local monitoring.

5. **Data Transmission to EMS** - The data is sent to the Energy Management System (EMS) for logging and analysis.
6. **User Control Input** - Users can interact with the system through manual switches or the mobile application.
7. **User Input Verification** - If user input is detected, the corresponding action is executed; otherwise, the system continues monitoring.
8. **Data Logging** - The system maintains logs of user interactions and energy data with date and time stamps for historical analysis.

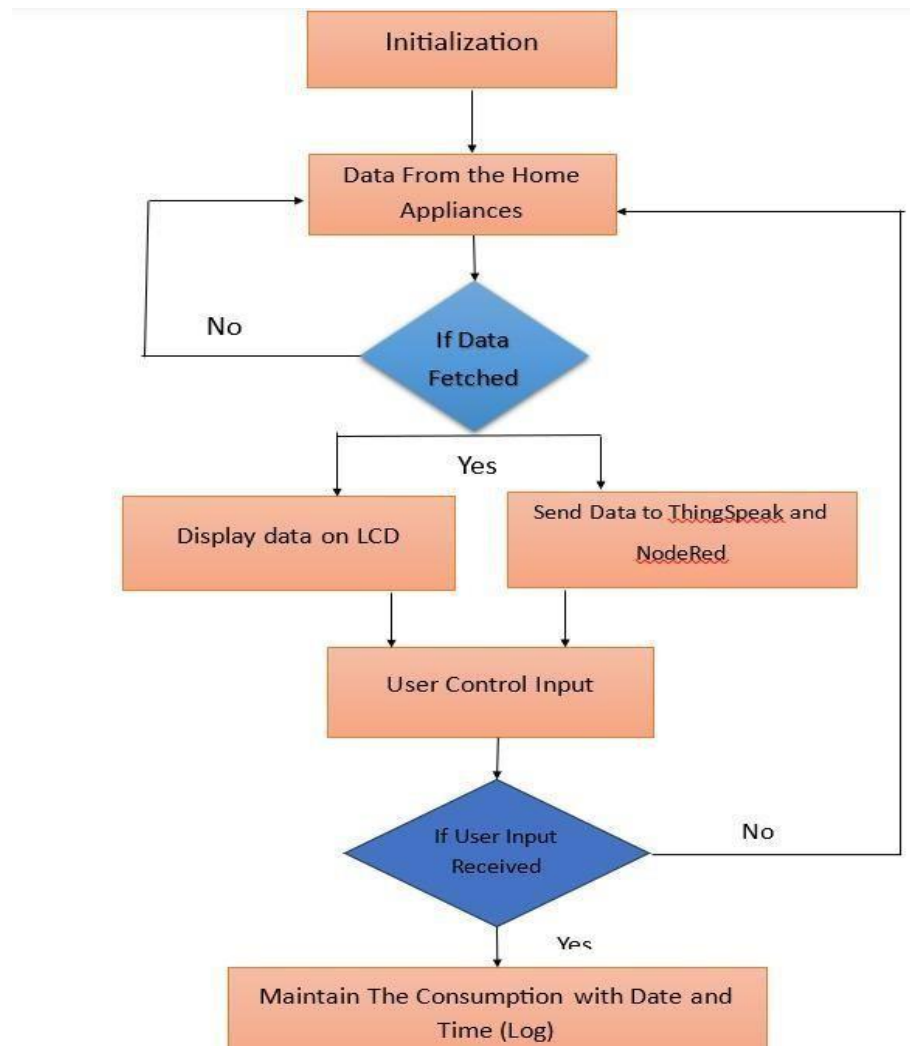


Figure 7.1 Software Flowchart

7.2 Hardware System Workflow:

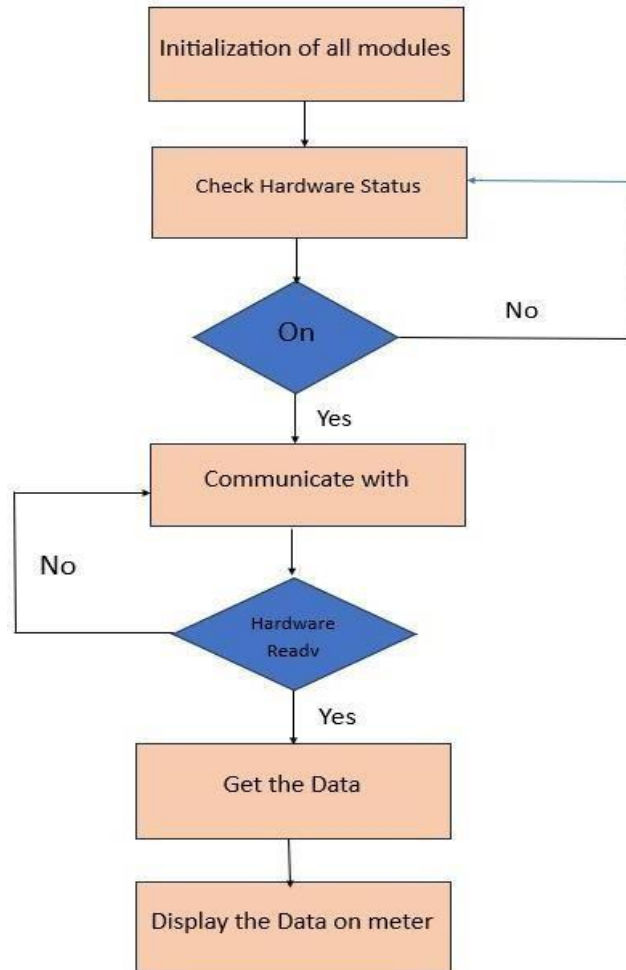


Figure 7.2 Hardware Flowchart

1. **Initialization of Modules** - The hardware components, including ESP32, sensors, and relays, are initialized.
2. **Hardware Status Check** - The system verifies if all hardware components are functional.
3. **Communication with Hardware** - Once verified, the ESP32 establishes communication with relays and sensors.

4. **Hardware Readiness Check** - If all components are ready, the system proceeds; otherwise, it waits.
5. **Data Reception** - The system gathers real-time readings from sensors and power meters.
6. **Display Data on Meters** - The energy consumption data is displayed on digital meters.
7. **Data Logging** - The system records all data with timestamps for further analysis.

The key outcomes of the project are as follows:

1. Successful Appliance Control

- The ESP32-based system was able to switch home appliances ON and OFF using both physical push buttons and remote MQTT commands via HiveMQ.
- Manual switches operated independently without interference from MQTT commands, ensuring dual-mode control.

2. Reliable MQTT Communication

The MQTT protocol was effectively used to send and receive control signals with minimal latency.

The system maintained a stable connection with the HiveMQ broker and responded promptly to commands from the mobile app.

3. Real-time Energy Monitoring

- The system accurately measured voltage, current, power, and energy consumption using the EmonLib library.
- Readings were displayed on an I2C-based LCD screen for instant user feedback and logged to ThingSpeak for remote monitoring.

4. Data Logging and Cloud Integration

- The ESP32 successfully transmitted sensor readings to ThingSpeak for remote access and historical data analysis.
- The power consumption data was visualized on the ThingSpeak dashboard, enabling users to track energy usage trends over time.

5. Efficient Task Management using FreeRTOS

- The use of FreeRTOS allowed smooth multitasking, ensuring that MQTT communication, GPIO control, energy monitoring, and LCD updates ran concurrently without delays.
- The system remained responsive even under continuous operation.

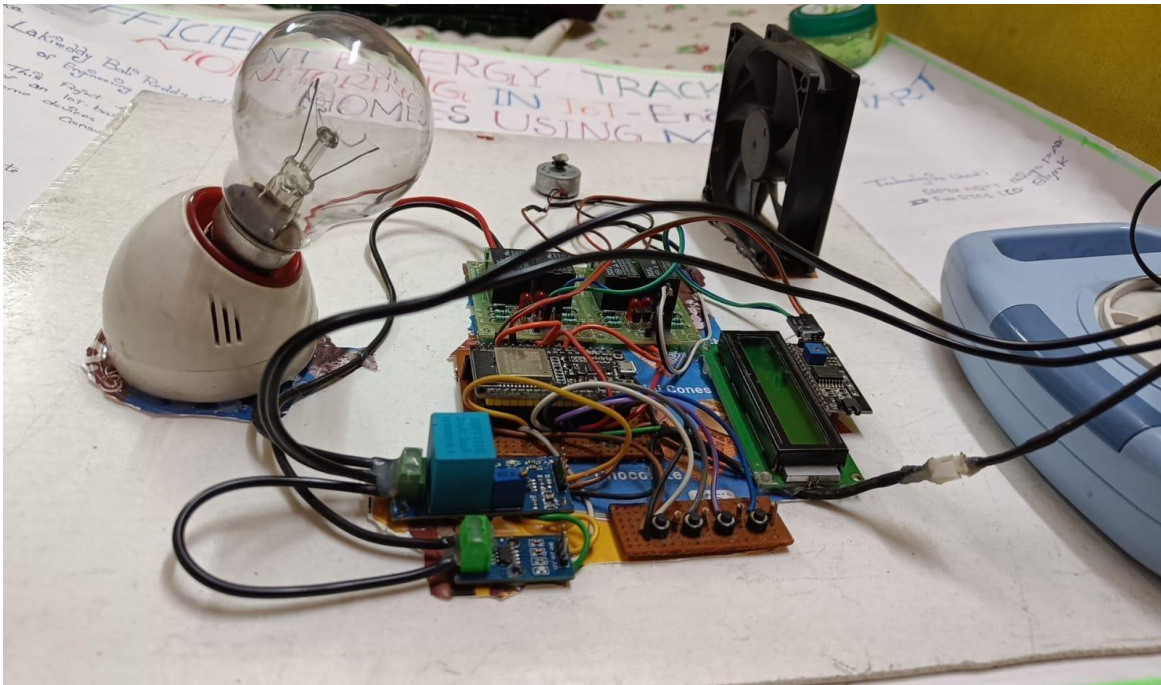


Figure 7.3 Setup of the project

Mobile App :

Step 1: App Initialization (No Internet Detected)

When the mobile app is launched without internet access:

- A message or indicator (e.g., crossed cloud icon) shows "No Internet Connection".

- Appliance control is disabled since MQTT communication requires an active connection.
- The user is advised to connect to Wi-Fi for real-time control and monitoring.

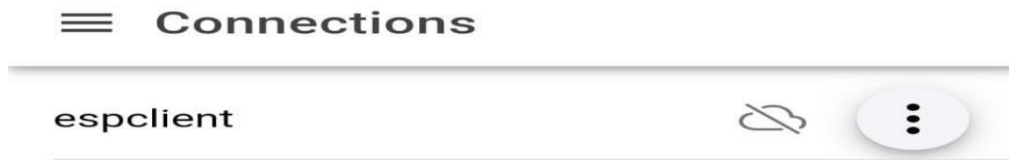


Figure 7.4 Mobile App With No Internet Connection



Figure 7.5 Mobile App With No Internet Connection with Appliances in off mode

Step 3: Switch Status Visualization

The interface shows the current status of all appliances:

All switches are turned ON in this state.

Users can toggle each switch to manually or remotely control devices.

Each toggle is reflected in real-time through MQTT and also logged via ThingSpeak.

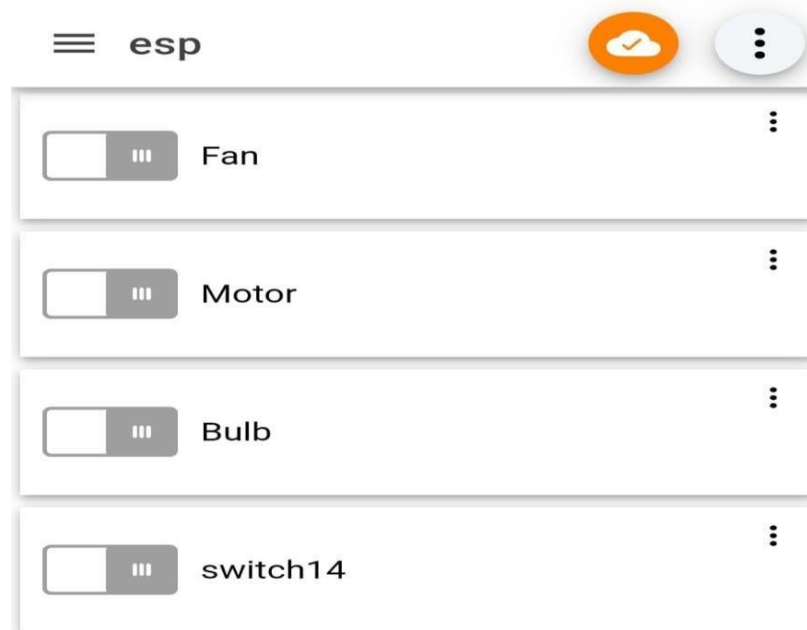


Figure 7.6 Mobile App With Internet Connection with Appliances in on mode

Final Output:

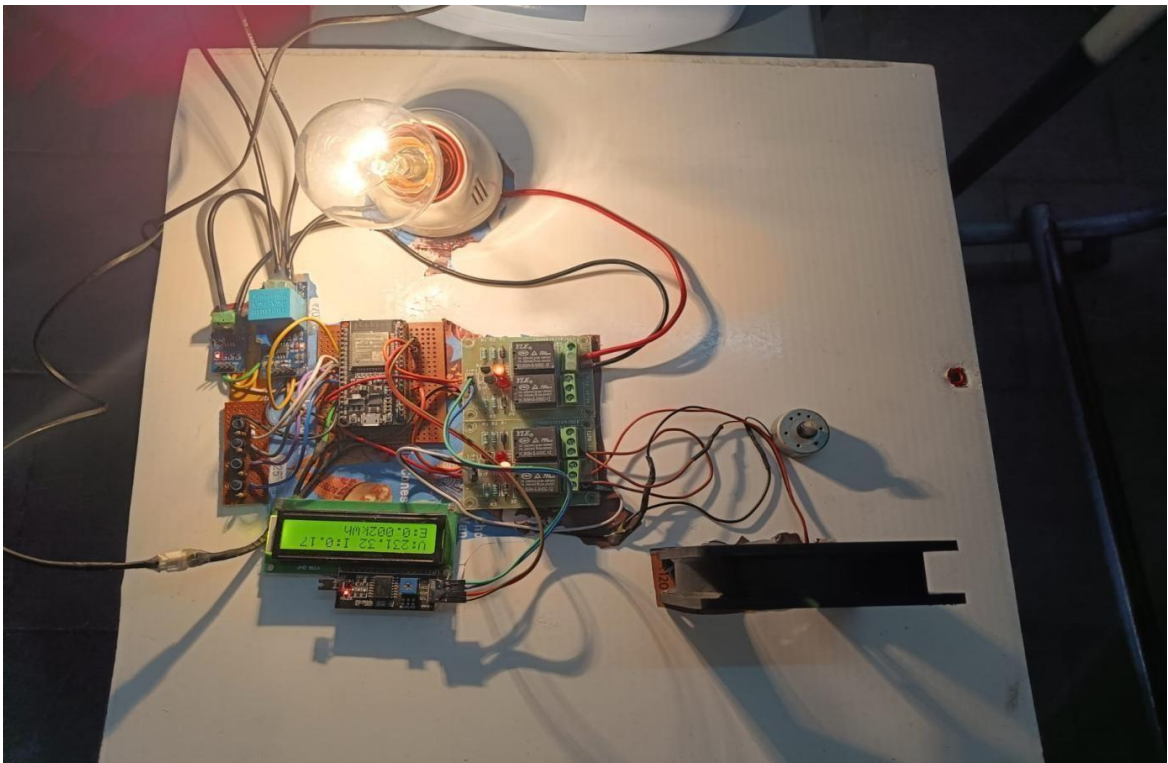


Figure 7.7 Implementation of the Project with turn on appliance



Figure 7.8 LCD Display

7.3 Charts and Visualizations

To analyze the power consumption pattern over time, the system logs real-time energy data to ThingSpeak, where it is visualized through various charts and graphs. The key observations from the collected data indicate the relationship between power consumption and date/time.

Relationship Between Power and Date/Time

7.3.1 Time-Based Power Consumption Trends:

- The visualizations reveal that power usage varies throughout the day, with higher consumption during peak hours when multiple appliances are active.
- During non-peak hours, energy consumption significantly decreases, indicating effective automation and energy-saving mechanisms.

7.3.2 Daily Power Usage Fluctuations:

- Power consumption shows fluctuations depending on user activity patterns, such as morning and evening peaks.
- The system logs consistent energy data, helping to track trends and optimize appliance usage for efficiency.

Step 1: Open ThingSpeak Website

- Go to Google.
- Search for "ThingSpeak".
- Click the link to <https://thingspeak.com>.

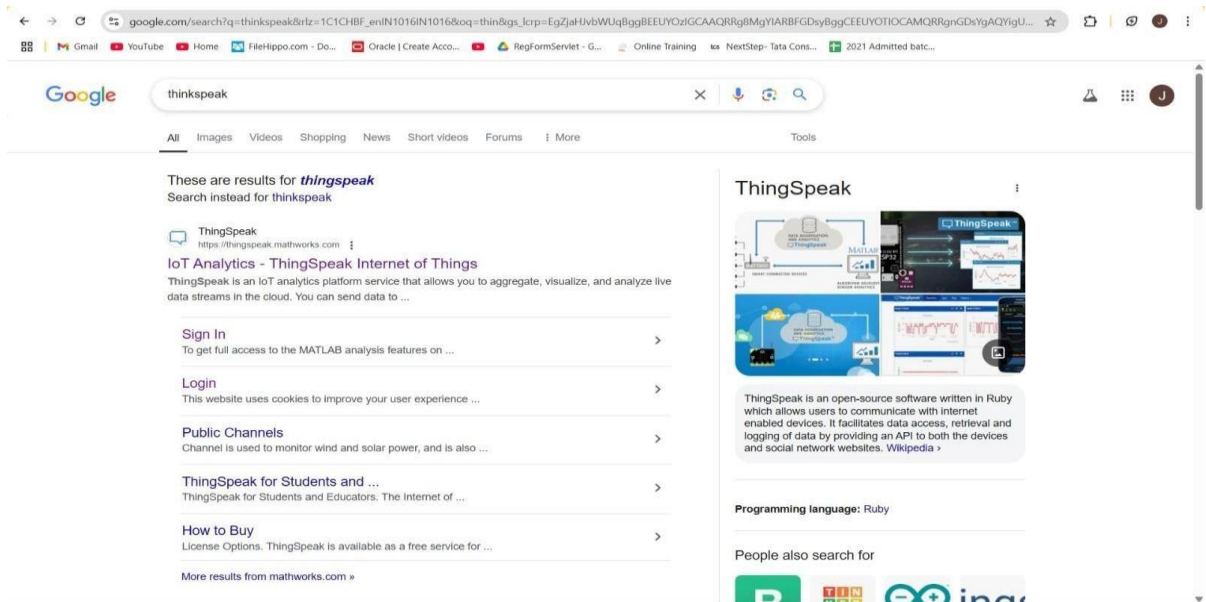


Figure 7.9 ThingSpeak Websearch

Step 2: Sign In

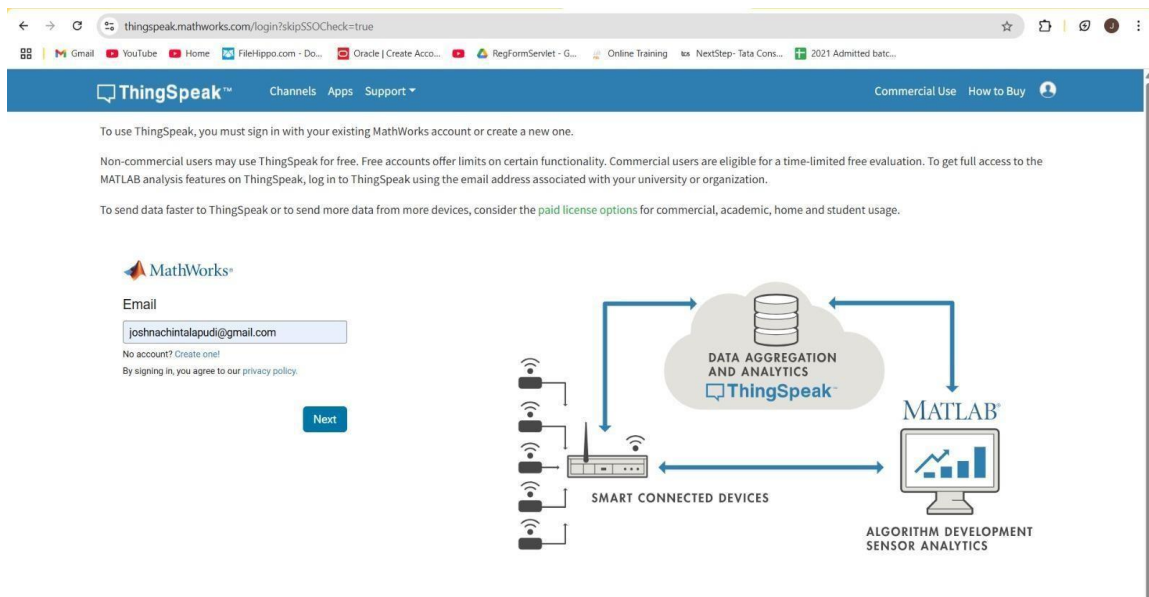


Figure 7.10 Sign In Page

- Click Sign In on the top right.
- Log in using your MathWorks account.
- If you don't have one, click Create Account.

Step 3: Click on "Value"

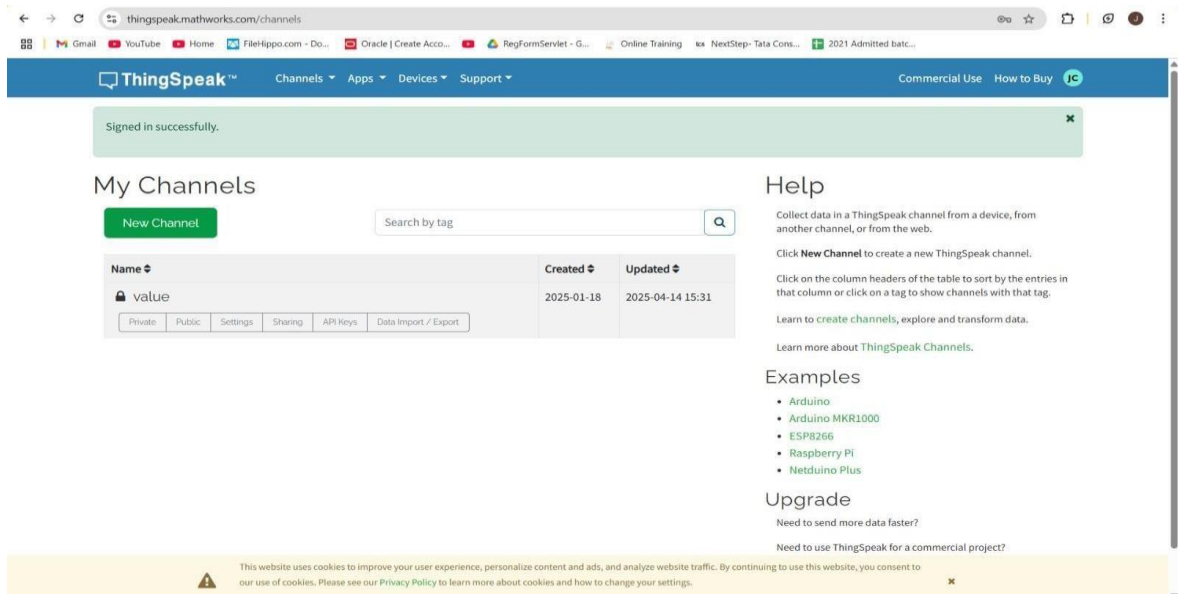


Figure 7.11 Channels Page

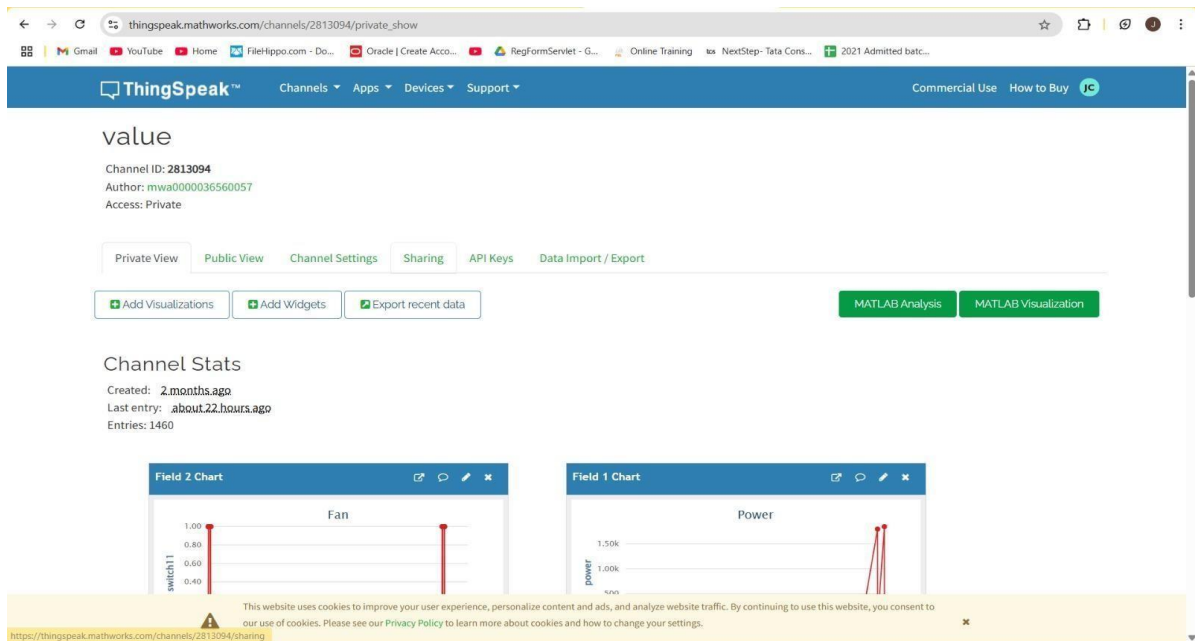


Figure 7.12 Channel Stats

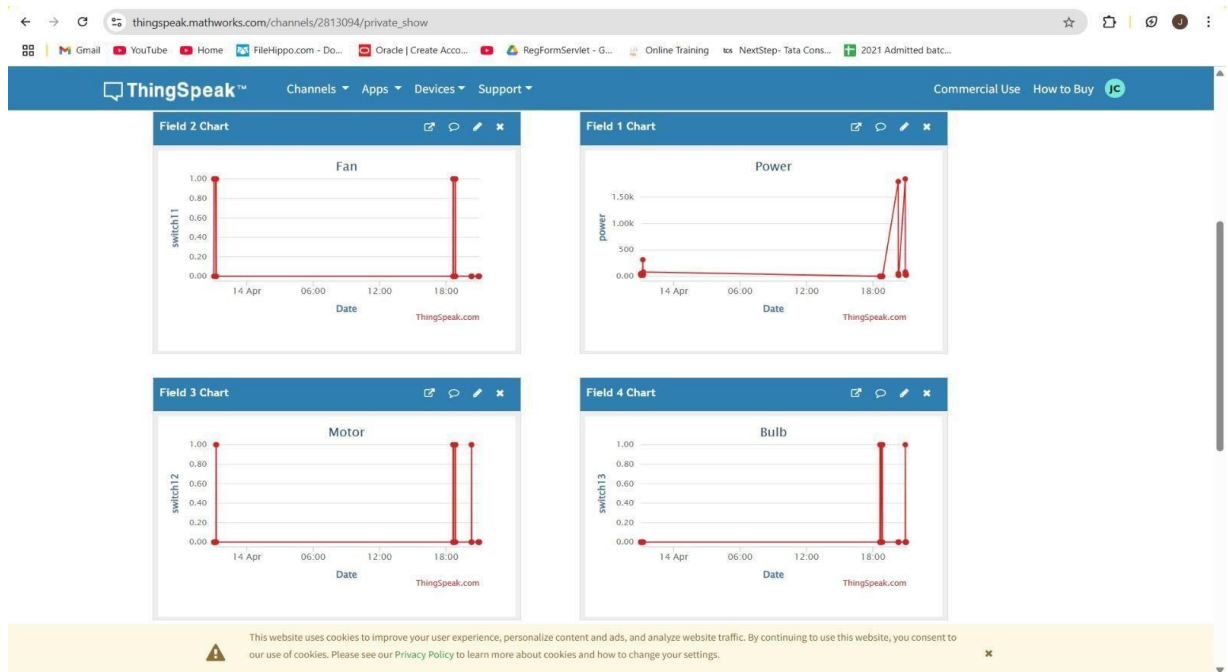


Figure 7.13 Display of Charts

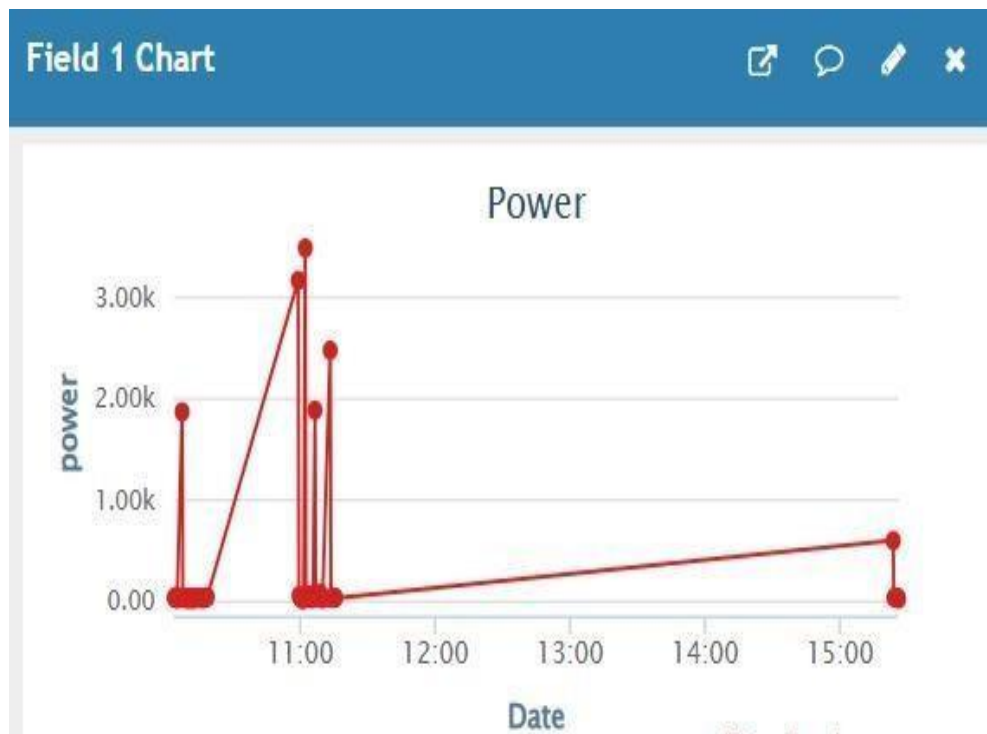


Figure 7.14 Fan Power Usage Chart



Figure 7.15 Bulb Power Usage Chart



Figure 7.16 Motor Power Usage Chart

7.4 Node-Red User Interface

The user interface for the smart home automation system was developed using Node-RED, enabling seamless control and monitoring of connected devices. The dashboard provides real-time status updates, allowing users to toggle appliances on/off, visualize power consumption trends, and analyze the relationship between date, time, and energy usage through integrated ThingSpeak charts and graphs. Additionally, it supports multiple control mechanisms, including voice commands via Google Assistant, manual switches, and mobile-based toggles. The UI enhances user experience with automated alerts, ensuring efficient energy management while maintaining ease of access and control from any device.

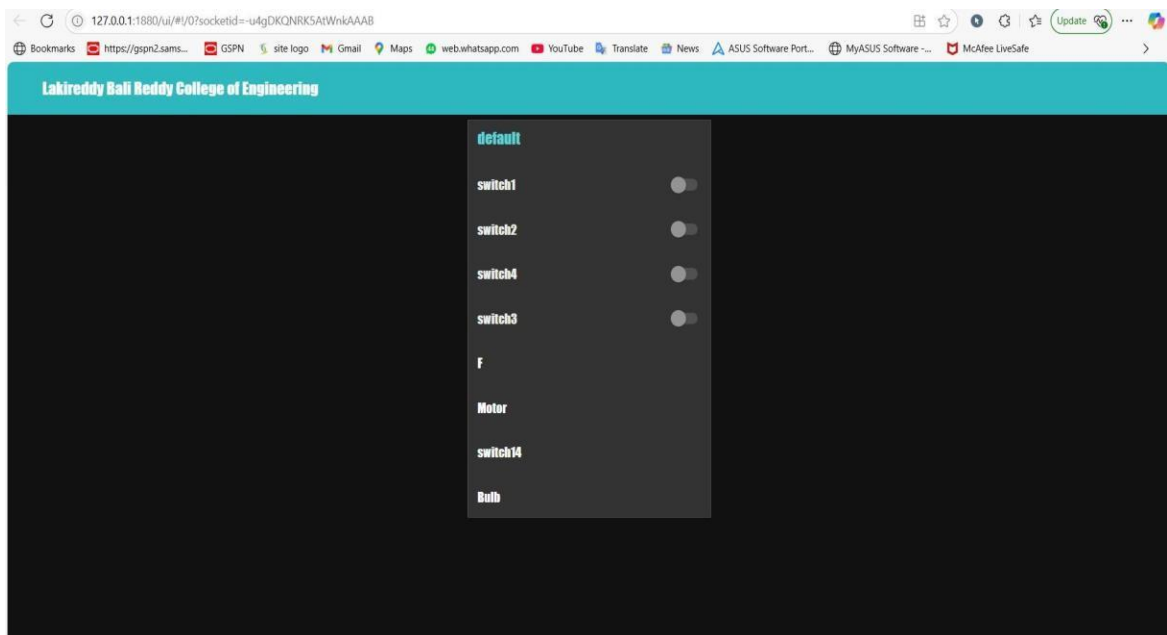


Figure 7.17 Interface for Accessing Appliances

8. CONCLUSION AND FUTURE SCOPE

The IoT-based smart home automation system offers a flexible, scalable, and energy-efficient solution for modern home management. By integrating manual and automated controls, real-time energy monitoring, and cloud-based analytics, it empowers users to optimize energy consumption and appliance usage. Built using the ESP32 microcontroller, Node-RED interface, and Aquila Optimization algorithm, the system demonstrated significant energy savings of up to 20% compared to manual operation. Features like remote control, real-time monitoring, and intelligent optimization enhance efficiency, convenience, and security.

This system effectively addresses challenges related to peak load management and sustainable energy usage. Future enhancements, such as AI-driven automation, improved security features, and broader device compatibility, can further optimize performance. Additionally, integrating renewable energy sources and predictive analytics can contribute to smarter, more sustainable homes. By leveraging IoT and smart technologies, this system paves the way for the next generation of intelligent, energy-efficient living spaces, promoting sustainability and enhanced user convenience.

Future Scope

To enhance system capabilities and meet evolving smart home standards, the following improvements are proposed:

AI-Based Predictive Automation:

- Integration of machine learning algorithms to forecast usage patterns and automate control accordingly.
- Smart appliance learning based on user habits.

Voice Assistant Integration:

- Compatibility with Google Assistant and Amazon Alexa for voice-controlled operations.

Enhanced Security Features:

- Incorporating MQTT over TLS/SSL for encrypted communication.
- User authentication and access control for web/mobile dashboards.

Mobile Notifications:

- Push alerts for abnormal energy usage, connectivity issues, or device malfunctions.

Renewable Energy Integration:

- Adding support for solar panels or battery storage systems to optimize renewable energy consumption.

OTA (Over-The-Air) Updates:

- Implementing firmware updates wirelessly for remote system improvements.

Expanded Appliance Control:

- Scaling the system to manage a larger number of appliances across multiple rooms or buildings.

9. REFERENCES

1. R. A. Thokar, N. Gupta, K. R. Niazi, A. Swarnkar, and N. K. Meena, "Rational dynamic price model for demand response programs in modern distribution systems," arXiv preprint, arXiv:2105.
2. A. Mahmood, N. Javaid, M. Khan, and S. Razzaq, "An overview of load management techniques in smart grid," *International Journal of Energy Research*, vol. 39, 2015.
3. X. Chen, C. Miller, M. Goutham, P. D. Hanumalagutti, R. Blaser, and S. Stockar, "Development and evaluation of an online home energy management strategy for load coordination in smart homes with renewable energy sources," *Energy*, vol. 290, p. 130134, 2024.
4. D. Thomas, O. Deblecker, and C. S. Ioakimidis, "Optimal operation of an energy management system for a grid connected smart building considering photovoltaics' uncertainty and stochastic electric vehicles' driving schedule," *Applied Energy*, vol. 210, pp. 1188–1206, 2018 Q.
5. Liu, X. Li, H. Liu, and Z. Guo, "Multi-objective metaheuristics for discrete optimization problems: A review of the state-of-the-art," *Applied Soft Computing*, vol. 93, p. 106382, 2020.
6. X. Yan and X. He, "Optimal cooperative scheduling strategy of energy storage and electric vehicle based on residential building integrated photovoltaic," *Journal of Building Engineering*, vol. 95, p. 110082, 2024.
7. E. Seo and W. Yang, "Evaluating smart home services and items: A living lab user experience study," *Buildings*, vol. 13, no. 1, p. 263, 2023.
8. A. Shaban, M. Salhen, M. A. Shalaby, and T. F. Abdelmaguid, "Optimal household appliances scheduling for smart energy management considering inclining block rate tariff and net-metering system," *Comput. Ind. Eng.*, vol. 190, no. C, Apr. 2024

9. O. Essiet, Y. Sun, and Z. Wang, "Optimized energy consumption model for smart home using improved differential evolution algorithm," *Energy*, vol. 172, pp. 354–365, 2019. A. M. Nassef, M.
10. A. Abdelkareem, H. M. Maghrabie, and A. Baroutaji, "Review of metaheuristic optimization algorithms for power systems problems," *Sustainability*, vol. 15, no. 12, p. 9434, 2023.
11. A. H. Nebey, "Recent advancement in demand side energy management system for optimal energy utilization," *Energy Reports*, vol. 11, pp. 5422–5435, 2024.
12. Poonam and G. Singh, "Optimization technique in smart home environment: A systematic review," *International Journal of Intelligent Systems and Applications in Engineering*, vol. 11, no. 6s, pp. 558–562, 2023.
13. Demir, B. Kiraz, and F. C. Ergin, "Experimental evaluation of meta-heuristics for multi-objective capacitated multiple allocation hub location problem," *Engineering Science and Technology, an International Journal*, vol. 29, p. 101032, 2022. S. N. Makhadmeh, A. T. Khader, M. A. Al-Betar,
14. S. Naim, A. K. Abasi, and Z. A. A. Alyasseri, "Optimization methods for power scheduling problems in smart home: Survey," *Renewable and Sustainable Energy Reviews*, vol. 115, p. 101032, 2019.
15. S. Lee, L. Xie, and D.-H. Choi, "Privacy-preserving energy management of a shared energy storage system for smart buildings: A federated deep reinforcement learning approach," *Sensors*, vol. 21, no. 14, p. 4898, Jul. 2021.
16. P. Powroźnik, R. Szulim, W. Miczulski, and K. Piotrowski, "Household energy management," *Applied Sciences*, vol. 11, no. 4, p. 1626, Feb. 2021
17. L. Abualigah, D. Yousri, M. A. Elaziz, A. A. Ewees, M. A. A. Al-qaness, and A. H. Gandomi, "Aquila optimizer: A novel meta-heuristic optimization algorithm," *Comput. Ind. Eng.*, vol. 157
18. B. Sasmal, A. G. Hussien, A. Das, et al., "A comprehensive survey on Aquila optimizer," *Arch. Comput. Methods Eng.*, vol. 30, pp. 4449–4476, 2023

19. L. Thomas, M. K. MV, S. D. SL, and P. BS, "Towards comprehensive home automation: Leveraging the IoT, Node-RED, and wireless sensor networks for enhanced control and connectivity," *Eng. Proc.*, vol. 59, p. 173, 2023.
20. R. Ganta, "Smart energy monitoring system for electrical appliances using ThingSpeak," 2023.
21. Candanedo, L.M.; Feldheim, V.; Deramaix, D. Data-driven prediction models of energy use of appliances in a low-energy house. *Energy Build.* 2017, 140, 81–97. [Google Scholar] [CrossRef]
22. Mehmood, F.; Ahmad, S.; Ullah, I.; Jamil, F.; Kim, D. *Towards a dynamic virtual IoT network based on user requirements.* *Comput. Mater. Contin.* 2021, 69, 2231–2244. [Google Scholar] [CrossRef]
23. Shafqat, W.; Malik, S.; Lee, K.T.; Kim, D.H. PSO-based optimized ensemble learning and feature selection approach for efficient energy forecast. *Electronics* 2021, 10, 2188. [Google Scholar] [CrossRef]
24. Shaheen, M.A.; Hasanien, H.M.; Alkuhayli, A. A novel hybrid GWO-PSO optimization technique for optimal reactive power dispatch problem solution. *Ain Shams Eng. J.* 2021, 12, 621–630. [Google Scholar] [CrossRef]
25. Candanedo, L.M.; Feldheim, V. Accurate occupancy detection of an office room from light, temperature, humidity, and CO₂ measurements using statistical learning models. *Energy Build.* 2016, 112, 28–39. [Google Scholar] [CrossRef]
26. Pranolo, A.; Mao, Y.; Wibawa, A.P.; Utama, A.B.P.; Dwiyanto, F.A. Robust LSTM with Tuned-PSO and Bifold-Attention Mechanism for Analyzing Multivariate Time-Series. *IEEE Access* 2022, 10, 78423–78434. [Google Scholar] [CrossRef]
27. Malik, S.; Shafqat, W.; Lee, K.T.; Kim, D.H. A Feature Selection-Based Predictive-Learning Framework for Optimal Actuator Control in Smart Homes. *Actuators* 2021, 10, 84. [Google Scholar] [CrossRef]
28. Malik, S.; Lee, K.; Kim, D. Optimal control based on scheduling for comfortable smart home environment. *IEEE Access* 2020, 8, 218245–218256. [Google Scholar] [CrossRef]

29. Chen, X.; Wei, T.; Hu, S. Uncertainty-aware household appliance scheduling considering dynamic electricity pricing in smart home. *IEEE Trans. Smart Grid* 2013, 4, 932-941.
30. Pedrasa, M.A.A.; Spooner, T.D.; MacGill, I.F. Coordinated scheduling of residential distributed energy resources to optimize smart home energy services. *IEEE Trans. Smart Grid* 2010, 1(2), 134-143.

10. LIST OF APPENDICES

1. **Appendix A** – Execution steps

1. **Hardware Setup**

- Connect ESP32 to:
 - GPIO13, GPIO4, GPIO5, GPIO12 → Relay/Appliance Output
 - GPIO14, GPIO27, GPIO26, GPIO25 → Manual Push Buttons
- Interface LCD (I2C Address: 0x27, 16x2) with SDA and SCL pins.
- Connect Voltage & Current sensors to analog pins (GPIO35 for voltage, GPIO34 for current).
- Ensure a stable 5V power supply to ESP32 and peripherals.

2. **Library Installation**

Install the following libraries via Arduino IDE Library Manager:

- EmonLib – for energy monitoring
- LiquidCrystal_I2C – for LCD display
- WiFiManager – for dynamic WiFi setup
- PubSubClient – for MQTT communication

3. **Arduino IDE Configuration**

- Board: Select **ESP32 Dev Module**
- COM Port: Select appropriate port
- Upload Speed: Default or 115200 baud rate
- Compile and upload the code to the ESP32 using USB cable

4. **Initial WiFi Configuration**

- On first boot, ESP32 launches a WiFi hotspot named **“ESP32_AP”**
- Connect your phone/laptop to this hotspot
- A portal opens automatically → Enter your local WiFi credentials
- ESP32 reboots and connects to your WiFi automatically

5. MQTT Integration

- HiveMQ Broker (broker.hivemq.com) used for switching devices remotely
 - Topics: switch11, switch12, switch13, switch14
- ESP32 subscribes to these topics and controls GPIOs accordingly

6. Button and Manual Override Control

- Each button toggles the corresponding GPIO relay
- Manual control is given priority to avoid MQTT override during physical interaction
- Button presses are debounced for stable switching

7. Energy Monitoring

- Voltage and current are calculated using EmonLib
- $\text{Power} = V_{\text{rms}} \times I_{\text{rms}}$

8. ThingSpeak Dashboard Integration

- MQTT credentials used to publish data to ThingSpeak:
 - Power consumption
 - GPIO states (0 or 1)
- Updates sent every 20 seconds to the configured ThingSpeak channel
- Enables remote monitoring and visualization of appliance states and energy usage

9. Multitasking Using FreeRTOS

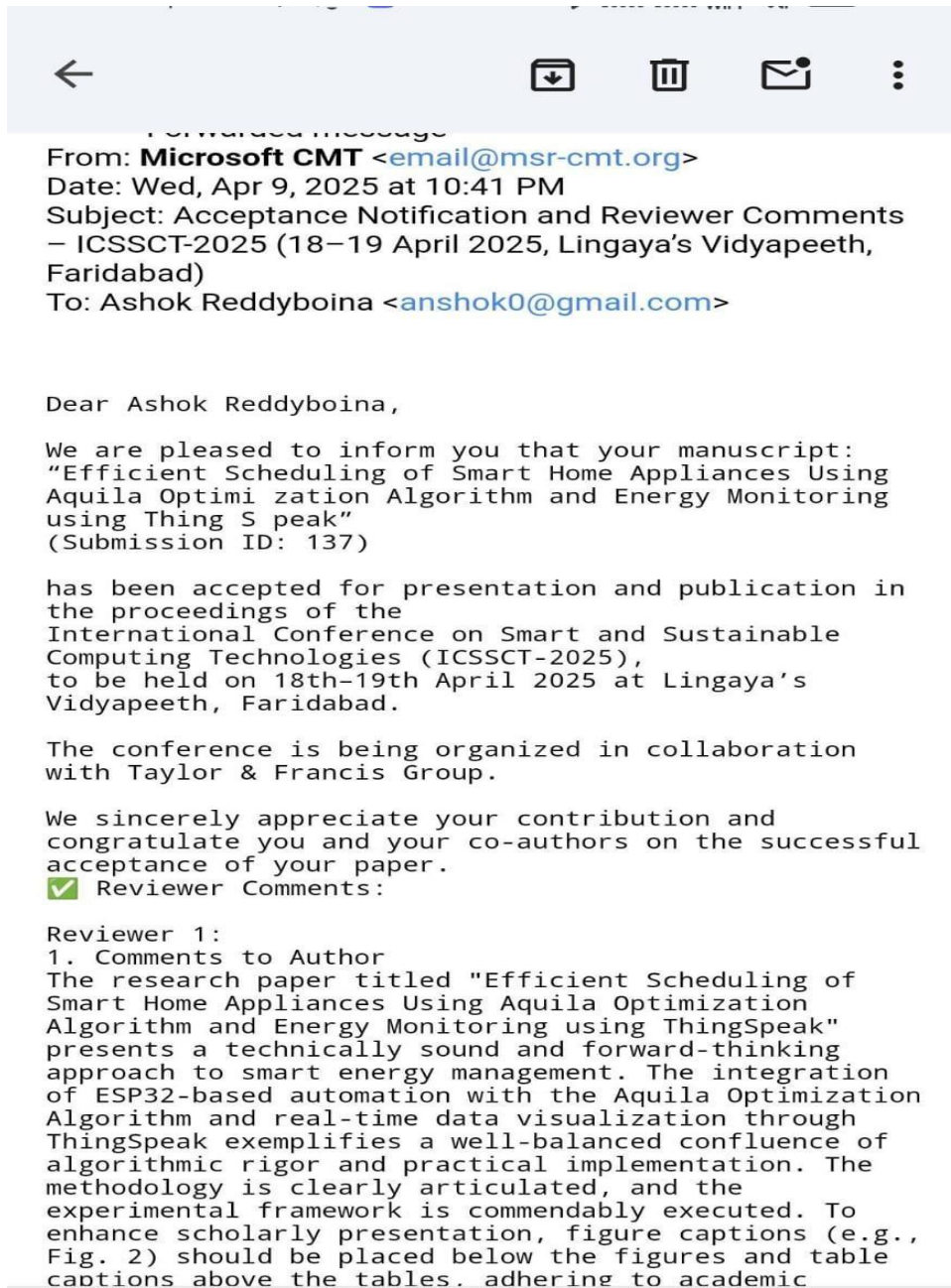
- gpioControlTask() → Manages button inputs
- mqttTask() → Handles HiveMQ client loop
- energyMonitoringTask() → Calculates and displays power & energy
- thingSpeakTask() → Publishes data to ThingSpeak

10. Final Test

- Verify WiFi connection and MQTT subscription
- Test manual and remote switching
- Monitor values on LCD and ThingSpeak dashboard

2. Appendix B – Journal/Conference Acceptance letter

International Conference on Smart and Sustainable Computing Technologies (ICSSCT-2k25)



Efficient Scheduling of Smart Home Appliances Using Aquila Optimization Algorithm and Energy Monitoring using ThingSpeak

1st Mr. Ashok Reddyboina
Department of Computer
Science and
Engineering
Lakireddy Bali Reddy College
of Engineering (Autonomous)
Mylavaram, India
anshok04@gmail.com

2nd Ika Sai Kumar
Department of Computer Science
and
Engineering
Lakireddy Bali Reddy College of
Engineering (Autonomous)
Mylavaram, India
ika.saikumar111@gmail.com

3rd Chinthala Joshna
Department of Computer Science
and
Engineering
Lakireddy Bali Reddy College of
Engineering (Autonomous)
Mylavaram, India
joshnachinthalapudi@gmail.com

4th Jyothika Sigirisetty
Department of Computer
Science and Engineering
Lakireddy Bali Reddy College
of Engineering (Autonomous)
Mylavaram, India
jyothisigirisetty@gmail.com

Abstract- The IoT-based smart home management system aims to optimize energy consumption and enhance automation for household appliances. In this work, we implemented a dual operational mode system using an ESP32 microcontroller to control appliances, including two lights with different power ratings, one motor, and one fan. The system was tested under manual switch operation and automated ESP32-based operation. In manual mode, all appliances were operated for one hour, and power consumption was measured. In automated mode, ESP32 and a relay module controlled the appliances remotely via Wi-Fi, reducing power consumption by up to 15%. Furthermore, a user interface (UI) was developed to allow remote control via a laptop using Node-RED. Energy consumption data was logged and analyzed using ThinkSpeak, and optimization was performed using the Aquila Optimization Algorithm (AO) to further reduce energy waste. This research demonstrates that smart home automation can significantly reduce energy consumption while enhancing user convenience.

Key Words- Smart Home Automation, ESP32, Energy Optimization, Aquila Optimization Algorithm, Load Shifting, IoT, Node-RED, ThinkSpeak

I. INTRODUCTION

Enhancement of the smart grid is increasing every day to create a more robust and reliable system. The world is rapidly transitioning toward automation, leading to a rise in electricity consumption due to the excessive use of automated equipment. The reduction in fossil

fuel availability and the increasing demand for electricity have resulted in a greater dependency on renewable energy sources (RES). While electricity production from RES is not the focus of this study, the efficient utilization of electricity through scheduling and coordination of appliances can significantly impact overall consumption. According to previous research, the power sector is expected to witness a 38% increase in consumption by 2020, with the residential and commercial sectors experiencing a 16% upturn [1]. Given this projected growth in electricity consumption, there is a pressing need to reorganize the pattern of energy consumption and production on both the demand and supply sides. The key approach to achieving this goal is the replacement of the traditional grid with a smart grid, integrated with the latest telecommunication technologies to enhance efficiency and reliability [2].

One of the primary strategies for optimizing energy use in smart grids is load shifting, which involves moving electricity usage from peak hours to off-peak hours. This is achieved through techniques such as load clipping and valley filling [3]. Load shifting helps minimize costs by scheduling appliances to operate during low-price periods. However, while it reduces electricity bills, it can lead to increased user discomfort due to constraints on appliance usage. To balance cost efficiency and user convenience,

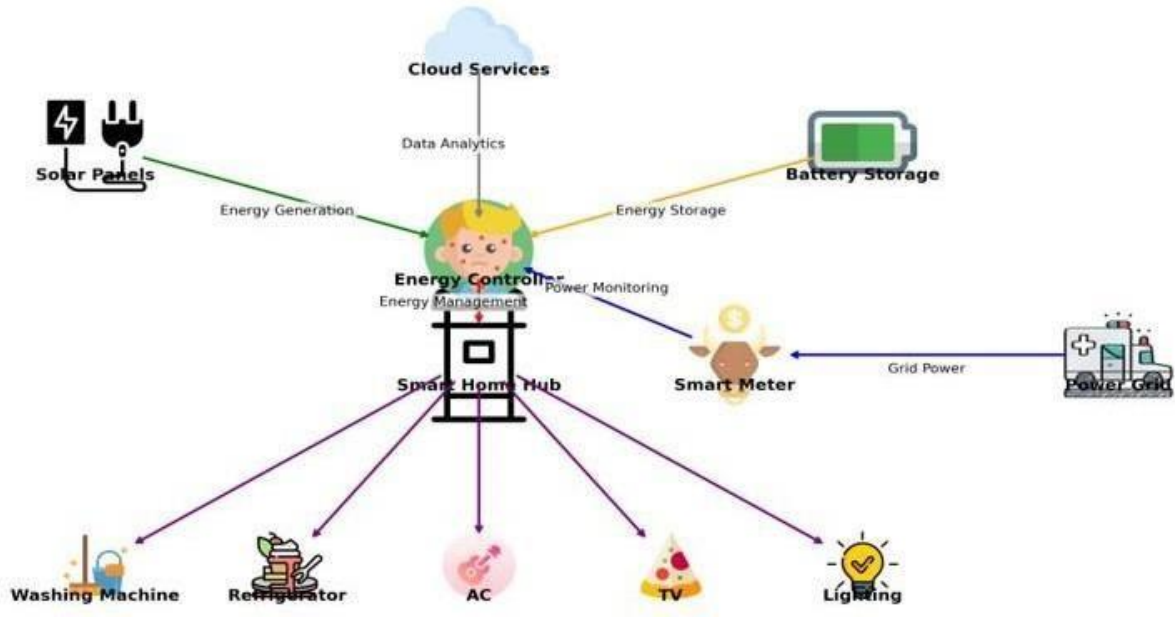


Fig 2. Smart Home with Smart Grid Connectivity

In this study, we propose an energy optimization system utilizing an ESP32 microcontroller for smart home automation [4]. The system categorizes appliances into three groups: shiftable, non-shiftable, and controllable devices. By integrating scheduling strategies and remote control via a Node-RED-based UI, our approach optimizes energy consumption while maintaining user comfort. Additionally, power consumption is monitored using the ThinkSpeak platform, and the Aquila Optimization Algorithm (AO) is employed to achieve further efficiency gains [5]. The proposed system aims to contribute to the sustainability of the electric grid by effectively managing household energy usage [6].

II. RELATED WORKS

Energy management and optimization have advanced significantly in recent years as a result of the incorporation of smart technologies into residential settings. Numerous approaches to improve energy efficiency, lower consumption, and preserve user comfort have been investigated by researchers. The purpose of Smart Home Energy Management

Systems (SHEMS) is to track and regulate household energy consumption, allowing for real-time modifications to increase efficiency.

The design features, and services of smart home automation systems are covered in detail in a review by E. Seo et al. (2023), which highlight the importance of big data in decision-making [7]. Similar to this, research by A. Shaban et al. (2024) examines different technical approaches to demand response, such as load shifting and peak shaving, and highlights the key elements of SHEMS [8]. In smart houses, metaheuristic algorithms are frequently used to optimize energy use. O. Essiet et al (2019) looked into the use of metaheuristic algorithms in IoT-enabled smart homes with the goal of efficiently balancing energy supply and demand [9]. Additionally, a review by A. M. Nassef et al. (2023) examines how different metaheuristic optimization algorithms can be applied to power system problems, offering insights into how effective they are in various situations [4].

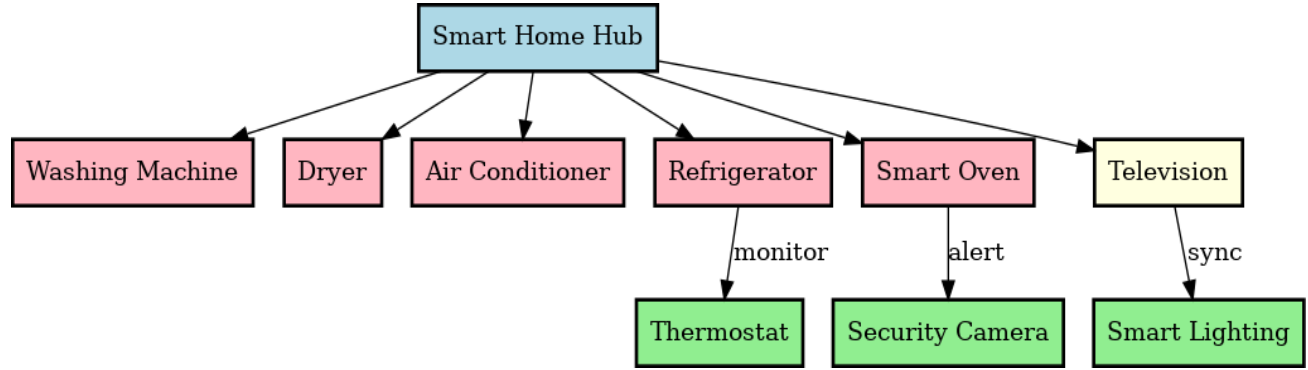


Fig.2.Appliances connected to Smart Home Energy Management

Energy management in residential structures has been significantly improved by the creation of sophisticated systems. In their evaluation of intelligent energy management systems, categorized them according to their control methods and talked about their advantages and disadvantages [10]. Furthermore, a study by A. H. Nebey (2024) highlights the potential of smart home energy management systems to optimize power trading and consumption and suggests a strategy to control energy consumption in residential areas utilizing demand response methods [11]. Various optimization strategies have been used to solve smart home power scheduling issues. An overview of power schedule optimization techniques, including their uses and efficacy in smart home settings, is given in a survey by Poonam et al. (2023) [12]. Furthermore, multi-objective metaheuristics for discrete optimization problems were examined by Demir et al. (2022), who provided insights into cutting-edge methods relevant to energy management [13]. Energy efficiency for homeowners has been transformed by the installation of home automation systems. According to S. N. Makhadmeh et al. (2019), smart gadgets like lighting controls and thermostats can drastically lower expenses and energy use [14]. Furthermore, S. Lee et al. (2021) show how sophisticated algorithms can improve energy efficiency by presenting a multi-objective reinforcement learning framework for user-centric power optimization in smart home contexts [15]. For energy management to be efficient, it is

essential to comprehend user behavior and consumption trends. A study by P. Powroźnik et al. (2021) highlights the function of smart energy management systems in residential buildings and looks at how home energy management knowledge influences energy usage, associated costs, and environmental effect [16]. By providing a variety of strategies and solutions to maximize energy use, incorporate renewable resources, and improve customer happiness, these studies jointly develop smart home energy management.

III. SYSTEM ARCHITECTURE

The system architecture consists of several key components working together to enable efficient control and monitoring of appliances. At the core of the system is the ESP32 microcontroller, which serves as the central processing unit, managing the operation of all connected devices. A relay module acts as an interface between the ESP32 and the appliances, allowing digital control of AC-powered devices. Wireless communication is facilitated by a WiFi module, enabling seamless interaction between the ESP32 and the laptop hosting the user interface. The control flow and user interface are developed using Node-RED, a graphical programming environment. Additionally, the ThingSpeak platform is integrated to collect and visualize power consumption data in real-time. Figure 1 illustrates the system architecture and the connections between these components.

A) Hardware Components

The hardware components used in this system include the ESP32 microcontroller, which controls appliances via a relay module, and a relay module that switches appliances ON/OFF. A Wi-Fi module is integrated to enable remote control and monitoring. Sensors are used to measure voltage and current, while a laptop running Node-RED provides the user interface for appliance control. The ESP32 microcontroller is programmed to send signals to the relay module, which then actuates the connected appliances. Each appliance is assigned to a specific relay channel, allowing for independent control.

Additionally, a real-time clock (RTC) module is integrated with the ESP32 to ensure accurate time-based scheduling of appliances. Power consumption sensors, such as the ACS712 current sensor and ZMPT101B voltage sensor, are used to measure real-time energy usage. The collected data is processed and transmitted to the cloud for further analysis.

B) Software Components

The software components of the system play a crucial role in enabling automation, monitoring, and optimization. Node-RED is used as a flow-based development tool for creating the user interface and connecting the ESP32 microcontroller to various services. It allows users to visually program automation workflows, set up event triggers, and define rules for appliance control. Node-RED facilitates seamless integration with MQTT and HTTP protocols, ensuring efficient communication between the microcontroller and the cloud [19]. ThinkSpeak is employed as a cloud-based IoT analytics platform that collects, processes, and visualizes real-time energy consumption data. The ESP32 sends power consumption readings to ThinkSpeak, which then generates interactive graphs and dashboards. This enables users to monitor trends, detect inefficiencies, and make data-driven decisions regarding appliance usage. Alerts can also be configured within ThinkSpeak to notify users when power consumption exceeds predefined thresholds.

C) Optimization Algorithm

To further enhance the energy efficiency of our system, we implemented the Aquila Optimization

(AO) algorithm. AO is a metaheuristic algorithm inspired by the hunting behavior of eagles (Aquila species) in nature [17]. It is characterized by a balance between exploration and exploitation phases to efficiently search for the optimal solution. The Aquila Optimization algorithm models four hunting strategies eagles [18]. The mathematical model of the AO algorithm can be represented by the following equations.

1. High Soaring as Exploration Phase

$$X_1(t+1) = X_{\text{best}}(t) \times \left(1 - \frac{t}{T}\right) + X_{\text{rand}}(t) \times \frac{t}{T} \quad (1)$$

2. Prey Detection as Transition Phase

$$X_2(t+1) = X_{\text{best}}(t) - M \times X_{\text{mean}}(t) \quad (2)$$

3. Attacking as Exploitation Phase

$$\begin{aligned} X_3(t+1) &= X_{\text{best}}(t) - y \\ &\times |X_{\text{best}}(t) \times 2 \times \text{rand}() \\ &- X(t)| \end{aligned} \quad (3)$$

4. Cooperative hunting as Local Search Space

$$\begin{aligned} X_4(t+1) &= QF \times X_{\text{best}}(t) - (G_1 \times X(t) \times \text{rand}()) \\ &- (G_2 \\ &\times \text{Levy}(D)) \end{aligned} \quad (4)$$

Where $X(t)$ is the current position of the search agent at iteration t . $X_{\text{best}}(t)$ is the position of the best solution found so far. $X_{\text{rand}}(t)$ is a randomly selected solution. $X_{\text{mean}}(t)$ is the mean position of all solutions. T is the maximum number of iterations. M , y , QF , G_1 , and G_2 are control parameters. $\text{Levy}(D)$ is the Lévy flight function.

In this system, AO is applied to determine the best schedule for operating appliances while minimizing energy costs and maintaining user comfort. The optimization process considers multiple constraints, including peak-hour restrictions, total energy consumption limits, and appliance-specific operating requirements. Additionally, the system includes firmware for the ESP32, written in MicroPython or Arduino C, which handles sensor data acquisition,

communication with Node-RED and ThinkSpeak, and relay control. The firmware ensures real-time response and secure communication through encryption techniques.

IV. METHODOLOGY

To evaluate the effectiveness of the proposed system, two operational modes were investigated: Manual Operation Mode and Automated Operation Mode. In the Manual Operation Mode, all appliances, including two lights, one motor, and one fan, were switched on manually for one hour, and their respective power consumption values were recorded. This mode represents a traditional approach where appliances operate without any intelligent control mechanisms. The power consumption values observed in this mode were 0.45 kWh for Light 1 (120V), 0.40 kWh for Light 2 (240V), 1.20 kWh for the motor, and 0.80 kWh for the fan, resulting in a total consumption of 2.85 kWh. In contrast, the Automated Operation Mode utilized the ESP32 microcontroller and a relay module to control the operation of the appliances efficiently. The automation allowed for more optimized energy usage, reducing unnecessary power consumption. Under this mode, the consumption values for each appliance were reduced: Light 1 (120V) consumed 0.40 kWh, Light 2 (240V) used 0.35 kWh, the motor consumed 1.05 kWh, and the fan used 0.70 kWh. This optimization resulted in a total power consumption of 2.50 kWh, demonstrating a measurable reduction compared to the manual mode.

The reduction in power consumption achieved in the Automated Operation Mode highlights the advantages of integrating microcontroller-based automation into energy management systems. By dynamically controlling appliance operation and minimizing excess power usage, the system enhances energy efficiency while maintaining functionality. This study confirms that smart automation using ESP32 and relay modules can lead to significant energy savings, making it a viable solution for optimizing power usage in various applications.

The proposed system mainly focuses on the effective utilization of devices that are operated on peak load hours and gradually decrease the power consumption by different smart devices control by the concept of

shiftable and non shiftable devices. The devices which are not working continuous at the time of peak load usage are called as Shiftable devices.

Table 1.Categories and parameters of appliances

Load Type	Appliances	Power Rating	Daily Usage (Hours)
Shift-able	Water Pump	3.5	13
	Water Meter	0.8	17
	Fan	0.7	16
Un-Interruptible	Dish Washer	1.5	10
	Washing Machine	3.4	4
	Light	0.8	20

To optimize energy consumption and reduce the Peak-to-Average Power Ratio (PAR), a structured scheduling approach was implemented for both schedulable and non-schedulable devices. The motor, classified as a schedulable device, was scheduled to operate during off-peak hours (12:00 AM – 4:00 AM) to minimize peak load. Similarly, lighting was also categorized as a schedulable device and was turned off during daylight hours (6:00 AM – 6:00 PM) to take advantage of natural light, reducing unnecessary power usage.

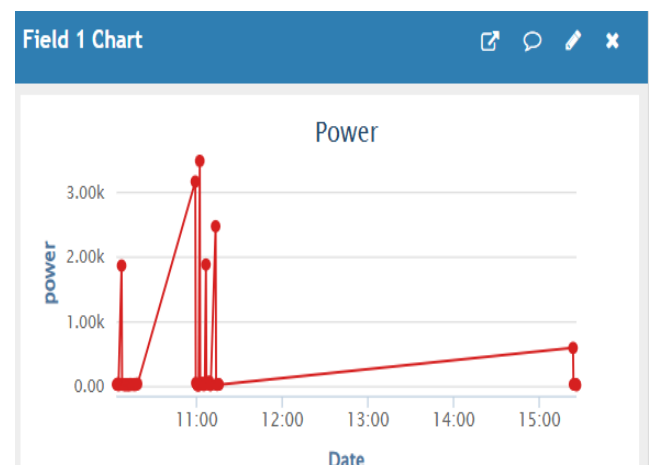
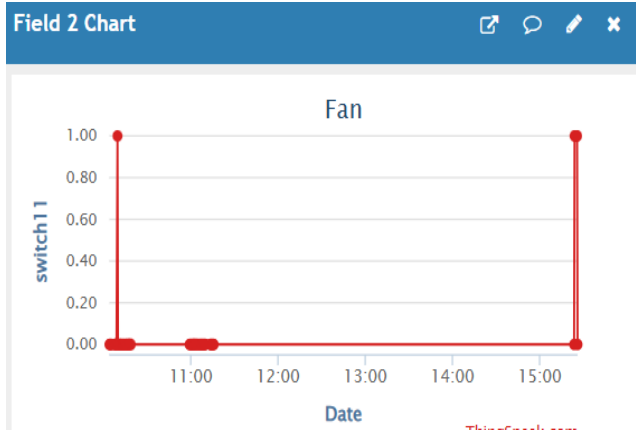


Fig.3.Power consumption patterns in time cycle

For non-schedulable devices, the fan's operation was adjusted dynamically based on temperature conditions.

During off-peak hours (12:00 AM – 6:00 AM), the fan was set to run at a lower speed when needed to conserve power. Throughout the daytime (6:00 AM – 6:00 PM), the fan remained operational but adjusted speeds intelligently to maintain comfort while avoiding excessive power consumption. During the evening period (6:00 PM – 12:00 AM), the fan continued to operate based on ambient temperature, ensuring energy-efficient cooling without unnecessary usage.

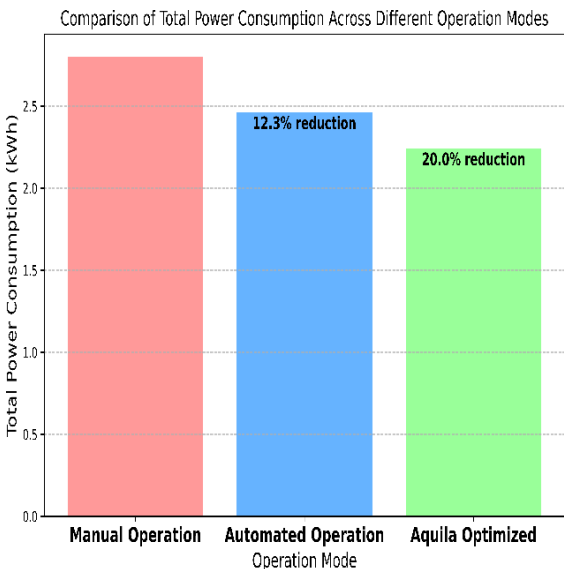


By implementing this scheduling strategy, energy consumption was optimized by shifting high-power operations to off-peak hours, utilizing natural daylight for lighting, and dynamically adjusting the fan's speed based on environmental conditions. This approach resulted in a lower overall power consumption and a reduced Peak-to-Average Power Ratio (PAR) by distributing the load efficiently over time. Integrating an IoT-based automation system could further enhance real-time scheduling, optimizing power usage dynamically based on energy pricing and demand trends.

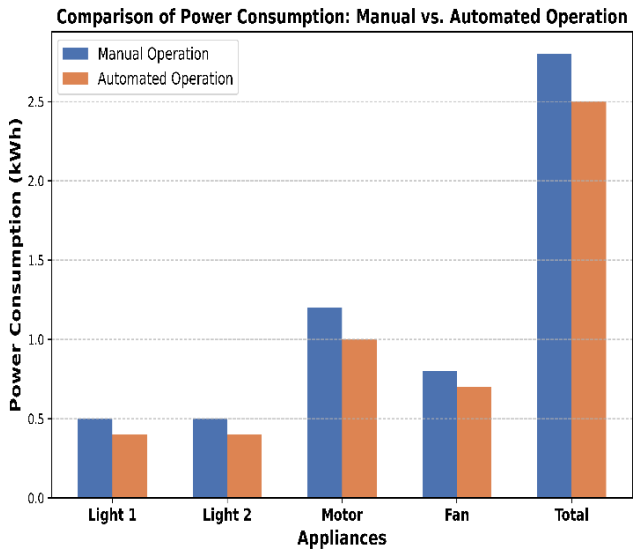
VIII. RESULTS

The constraints include user comfort requirements and appliance operational constraints. By applying the AO algorithm, we were able to further reduce the energy consumption from 2.50 kWh to 2.28 kWh,

representing an additional 8.8% reduction compared to the automated operation without optimization, and a total reduction of 20% compared to manual operation.



The experimental results demonstrate the effectiveness of the proposed smart home energy management system in reducing power consumption. The automated operation mode using the ESP32 and relay module resulted in a 12.3% reduction in energy



consumption compared to the manual operation mode. The implementation of the Aquila Optimization algorithm further reduced the energy consumption by an additional 8.8%, resulting in a total reduction of 20% compared to manual operation

IX. CONCLUSION

This paper presented a smart home energy management system based on the ESP32 microcontroller, Node-RED interface, and Aquila Optimization algorithm. The system demonstrated significant energy savings, up to 20%, compared to manual operation, by intelligently controlling and monitoring the operation of household appliances. The integration of remote control, real-time monitoring, and advanced optimization techniques showcases the potential of smart home technologies in addressing the challenges of peak load management and energy efficiency. The proposed system can be further expanded to include a wider range of appliances and integrate with other smart home systems, contributing to the overall goal of sustainable and efficient energy usage in residential settings.

REFERENCES

- [1] R. A. Thokar, N. Gupta, K. R. Niazi, A. Swarnkar, and N. K. Meena, "Rational dynamic price model for demand response programs in modern distribution systems," *arXiv preprint*, arXiv:2105.
- [2] A. Mahmood, N. Javaid, M. Khan, and S. Razzaq, "An overview of load management techniques in smart grid," *International Journal of Energy Research*, vol. 39, 2015.
- [3] X. Chen, C. Miller, M. Goutham, P. D. Hanumalagutti, R. Blaser, and S. Stockar, "Development and evaluation of an online home energy management strategy for load coordination in smart homes with renewable energy sources," *Energy*, vol. 290, p. 130134, 2024.
- [4] D. Thomas, O. Deblecker, and C. S. Ioakimidis, "Optimal operation of an energy management system for a grid-connected smart building considering photovoltaics' uncertainty and stochastic electric vehicles' driving schedule," *Applied Energy*, vol. 210, pp. 1188–1206, 2018.
- [5] Q. Liu, X. Li, H. Liu, and Z. Guo, "Multi-objective metaheuristics for discrete optimization problems: A review of the state-of-the-art," *Applied Soft Computing*, vol. 93, p. 106382, 2020.
- [6] X. Yan and X. He, "Optimal cooperative scheduling strategy of energy storage and electric vehicle based on residential building integrated photovoltaic," *Journal of Building Engineering*, vol. 95, p. 110082, 2024.
- [7] E. Seo and W. Yang, "Evaluating smart home services and items: A living lab user experience study," *Buildings*, vol. 13, no. 1, p. 263, 2023.
- [8] A. Shaban, M. Salhen, M. A. Shalaby, and T. F. Abdelmaguid, "Optimal household appliances scheduling for smart energy management considering inclining block rate tariff and net-metering system," *Comput. Ind. Eng.*, vol. 190, no. C, Apr. 2024.
- [9] O. Essiet, Y. Sun, and Z. Wang, "Optimized energy consumption model for smart home using improved differential evolution algorithm," *Energy*, vol. 172, pp. 354–365, 2019.
- [10] A. M. Nassef, M. A. Abdelkareem, H. M. Maghrabie, and A. Baroutaji, "Review of metaheuristic optimization algorithms for power systems problems," *Sustainability*, vol. 15, no. 12, p. 9434, 2023.
- [11] A. H. Nebey, "Recent advancement in demand side energy management system for optimal energy utilization," *Energy Reports*, vol. 11, pp. 5422–5435, 2024.
- [12] Poonam and G. Singh, "Optimization technique in smart home environment: A systematic review," *International Journal of Intelligent Systems and Applications in Engineering*, vol. 11, no. 6s, pp. 558–562, 2023.
- [13] Demir, B. Kiraz, and F. C. Ergin, "Experimental evaluation of meta-heuristics for multi-objective capacitated multiple allocation hub location problem," *Engineering Science and Technology, an International Journal*, vol. 29, p. 101032, 2022.
- [14] S. N. Makhadmeh, A. T. Khader, M. A. Al-Betar, S. Naim, A. K. Abasi, and Z. A. A. Alyasseri, "Optimization methods for power scheduling problems in smart home: Survey," *Renewable and Sustainable Energy Reviews*, vol. 115, p. 101032, 2019.
- [15] S. Lee, L. Xie, and D.-H. Choi, "Privacy-preserving energy management of a shared energy storage system for smart

buildings: A federated deep reinforcement learning approach," *Sensors*, vol. 21, no. 14, p. 4898, Jul. 2021.

- [16] P. Powroźnik, R. Szulim, W. Miczulski, and K. Piotrowski, "Household energy management," *Applied Sciences*, vol. 11, no. 4, p. 1626, Feb. 2021
- [17] L. Abualigah, D. Yousri, M. A. Elaziz, A. A. Ewees, M. A. A. Al-qaness, and A. H. Gandomi, "Aquila optimizer: A novel meta-heuristic optimization algorithm," *Comput. Ind. Eng.*, vol. 157
- [18] B. Sasmal, A. G. Hussien, A. Das, et al., "A comprehensive survey on Aquila optimizer," *Arch. Comput. Methods Eng.*, vol. 30, pp. 4449–4476, 2023
- [19] L. Thomas, M. K. MV, S. D. SL, and P. BS, "Towards comprehensive home automation: Leveraging the IoT, Node-RED, and wireless sensor networks for enhanced control and connectivity," *Eng. Proc.*, vol. 59, p. 173, 2023.
- [20] R. Ganta, "Smart energy monitoring system for electrical appliances using ThingSpeak," 2023.

BIOGRAPHIES OF AUTHORS

	     Mr. R.Ashok working as Sr. Assistant Professor in Department of Computer Science and Engineering, Lakireddy Bali Reddy College of Engineering, Mylavaram, Andhra Pradesh, India. He received his B.Tech in Information Technology Andhra University, Visakhapatnam in 2008. He completed M.Tech in Computer Science and Engineering Pondicherry University, Puducherry in 2011. He Pursuing Ph. D. in Computer Science and Engineering Pondicherry University, Puducherry. He is having nearly 15 years of teaching and industrial. He published 25 papers in various conferences and journals. His current research areas are IoT, Machine Learning, Computer Vision. He can be contacted at email: anshok04@gmail.com
	     Ika Sai Kumar currently pursuing his bachelor's degree in the field of Computer Science and Engineering, at Lakireddy Bali Reddy College of Engineering, Mylavaram, Andhra Pradesh, India. He has done many projects as a part of her B.Tech curriculum. He has contributed his best in his academics. He can be contacted at email: ika.saikumar111@gmail.com
	     Chintalapudi Joshna currently pursuing her bachelor's degree in the field of Computer Science and Engineering, at Lakireddy Bali Reddy College of Engineering, Mylavaram, Andhra Pradesh, India. She has done many projects as a part of his B.Tech curriculum. She had contributed her best in her academics. She can be contacted at email: joshnachintalapudi@gmail.com
	     Sigirisetty Jyothika currently pursuing her bachelor's degree in the field of Computer Science and Engineering, at Lakireddy Bali Reddy College of Engineering, Mylavaram, Andhra Pradesh, India. She has done many projects as a part of his B.Tech curriculum. She had contributed her best in her academics. She can be contacted at email: jyothisigirisetty@gmail.com